

Table des matières

Résumé	4
Abstract	5
Sigles et Abréviations	6
Glossaire	7
Liste des Figures	8
Liste des Tableaux	10
1 Généralités et État de l’art	14
1.1 Généralités	14
1.1.1 Généralités	14
1.2 État de l’art	16
1.2.1 SETI@home : calcul distribué volontaire à grande échelle	16
1.2.2 AWS Snowball : adaptation aux environnements à faible connectivité .	16
1.2.3 Amazon EC2 Spot Instances : réduction du coût du calcul	17
1.2.4 HTCondor : exploitation locale des ressources disponibles	17
1.3 Bilan et positionnement	17
1.3.1 Bilan	17
Étude de l’existant	17
1.3.2 Positionnement	18
2 Analyse et conception	19
2.1 Analyse	19
2.1.1 Analyse matérielle du cluster PARALLAX	19
Les machines DELL	19
Les mini-ordinateurs	21
Infrastructure réseau	22
Récapitulatif du matériel	23

	Limites matérielles par rapport au projet	24
2.1.2	Analyse logicielle	24
	Synthèse des intervenants et fonctionnalités	24
	Diagramme de contexte	25
	Fonctionnalités et exigences	25
	Présentation des cas d'utilisation	25
	Interaction Systemes	32
	Presentation des Etats Transition	37
2.2	Conception	37
2.2.1	Modélisation agentique du système	37
	Vue globale du SMA Parrallax	37
	Modélisation d'un agent	37
	Rôles des agents	38
	Définition des agents du monde SMA_PARALLAX	39
2.2.2	Interactions entre agents	42
	Table des événements du système	46
2.2.3	Architecture Agentique	48
2.3	Presentation des Algorithmes Cles	48
2.3.1	Présentation de l'algorithme d'inscription d'un nouveau nœud	48
2.3.2	Présentation de l'Algorithme d'élection du leader	49
	Principe	49
2.3.3	Objectif et déclencheurs	49
	Entrées	51
	Algorithme	51
	Sorties	51
2.3.4	Présentation de l'Algorithme de Décomposition des tâches et recomposition	52
2.3.5	Types de décomposition des calculs	53
	Décomposition par les données (Data Splitting)	53
	Décomposition par les instructions (Instruction Splitting)	53
2.3.6	Hypothèse de conception	53
2.3.7	Système d'annotations	54
2.3.8	Algorithme de Parsing	54
	Fonctionnement général du parser	55
2.3.9	Interprétation des annotations	55
2.3.10	Processus de décomposition et de recomposition des calculs	56
	Algorithme de décomposition des calculs	56
	Algorithme de recomposition des calculs	59
	Processus de recomposition des calculs	59

2.3.11	Presentation de l’algorithme de signalisation par heartbeat	60
	Rôles du heartbeat	60
	Architecture à deux couches	60
	Structure d’un message de heartbeat	61
	Paramètres temporels	61
	Détection de panne en deux phases	62
2.3.12	Modélisation UML	64
	Diagramme de classes métier	64
	diagramme de sequences Techniques	65
	diagramme d’etats d’une tache	69
	diagramme d’etats d’un noeud	70
2.3.13	Diagramme d’etats	71
2.3.14	Diagramme des composants	72
3	Implémentation et Résultats	73
3.1	Outils et Technologies	73
3.2	Diagramme de Déploiement	73
3.3	Résultats	73
	Conclusion Générale	73
	Références	73

Résumé

Dans un contexte où les chercheurs en mathématiques, biologie et physique réalisent des simulations complexes et entraînent des modèles d'intelligence artificielle, les besoins en puissance de calcul sont devenus considérables. À l'ENSPY, ces travaux reposent majoritairement sur des infrastructures en ligne, coûteuses et dépendantes d'une connexion Internet souvent instable, ce qui engendre des retards significatifs. Paradoxalement, de nombreuses machines disponibles sur le campus restent sous-utilisées. Face à cette situation, le projet PARALLAX a pour objectif de fédérer ces ressources existantes afin de mettre en place une infrastructure de calcul distribué assimilable à un super-calculateur local, permettant ainsi de réduire les coûts et d'améliorer les performances d'exécution. Dans cette optique, nous avons procédé à l'identification et au diagnostic des machines disponibles, à leur remise en état lorsque nécessaire, puis à leur interconnexion au sein d'un réseau local dont la connectivité a été validée.

Mots-clés : calcul distribué, super-calculateur, PARALLAX.

Abstract

In a context where researchers in mathematics, biology, and physics carry out complex simulations and train artificial intelligence models, the demand for computing power has become substantial. At ENSPY, these tasks rely mostly on online infrastructures, which are costly and dependent on an often unstable internet connection, leading to significant delays. Paradoxically, many machines available on campus remain underutilized.

Faced with this situation, the PARALLAX project aims to federate these existing resources in order to build a distributed computing infrastructure comparable to a local supercomputer, thereby reducing costs and improving execution performance.

In this context, we carried out the identification and assessment of the available machines, restored them when necessary, and then interconnected them within a local network whose connectivity has been validated.

Keywords : Distributed Computing, Super-Calculator, PARALLAX.

Sigles et Abréviations

Sigle	Signification
API	Application Programming Interface
AST	Abstract Syntax Tree
CLI	Command Line Interface
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DDR	Double Data Rate
DHCP	Dynamic Host Configuration Protocol
ENSPY	École Nationale Supérieure Polytechnique de Yaoundé
FIFO	First In, First Out
GB	Gigabyte
GHz	Gigahertz
IP	Internet Protocol
LAN	Local Area Network
LLM	Large Language Model
MAC	Media Access Control
MB	Megabyte
MPI	Message Passing Interface
RAM	Random Access Memory
RJ45	Registered Jack 45
SMA	Système Multi-Agent
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UUID	Universally Unique Identifier

Glossaire

- * **Annotation** : Directive spéciale insérée dans le code source par le développeur pour indiquer au système comment décomposer et distribuer une tâche (ex : `@parallax.split`).
- * **Contrôleur** : Nœud du cluster chargé de la supervision globale. Il reçoit les messages de heartbeat, vérifie l'état des machines en temps réel et détecte les pannes.
- * **Futex** : Mécanisme de synchronisation bas niveau permettant de bloquer un thread jusqu'à ce qu'une condition soit satisfaite, utilisé pour optimiser l'attente et la signalisation dans les systèmes concurrents.
- * **Gossip** : Méthode de communication où chaque nœud partage régulièrement ses informations avec quelques autres nœuds choisis au hasard, ce qui permet de diffuser rapidement les informations dans tout le cluster.
- * **Heartbeat** : Signal périodique émis par chaque nœud vers le contrôleur pour indiquer sa présence, son état et ses métriques de charge. Il permet la détection de panne et la supervision du cluster.
- * **Horloge logique de Lamport** : Méthode simple qui permet de mettre les événements dans un ordre cohérent dans un système distribué, même si les machines n'ont pas la même heure.
- * **Nœud** : Machine physique ou virtuelle appartenant au cluster PARALLAX. Chaque nœud exécute un agent autonome capable de communiquer, de traiter des tâches et de participer aux mécanismes distribués.
- * **Nœud maître (Master)** : Nœud du cluster chargé de la coordination globale. Il reçoit les programmes soumis, décompose les tâches, les distribue aux autres nœuds et agrège les résultats partiels.
- * **Nœud remplaçant** : Nœud de secours destiné à reprendre immédiatement le rôle du maître ou du contrôleur en cas de panne. Il possède une capacité intermédiaire et garantit la continuité du service.
- * **Nœud worker** : Nœud du cluster chargé d'exécuter les sous-tâches qui lui sont assignées par le maître et de renvoyer les résultats partiels.
- * **Quorum** : Nombre minimum de nœuds nécessaires pour valider une opération distribuée, notamment une élection, généralement défini comme $\lceil N/2 \rceil + 1$.

- * **SMA (Système Multi-Agent)** : Paradigme de modélisation dans lequel chaque entité (agent) est autonome, perçoit son environnement et prend des décisions localement. Dans PARALLAX, chaque nœud agit comme un agent.
- * **Split-brain** : Situation dans laquelle une partition du réseau conduit à l'existence simultanée de plusieurs leaders (maîtres), entraînant des incohérences dans le système.

Table des figures

2.1	Dell OptiPlex 380 — Machine 1	20
2.2	Dell OptiPlex 380 — Machine 2	20
2.3	Dell System 360 — Machine 3	21
2.4	Thin client Wyse — 533 MHz (×2)	22
2.5	Thin client Wyse — 1 GHz	22
2.6	Switch Ethernet 5 ports et câbles RJ45 sertis manuellement	23
2.7	Diagramme de Contexte	25
2.8	Diagramme de cas d'utilisation	26
2.9	Diagramme de séquences système — Importer une tâche	32
2.10	Diagramme de séquences système — Soumettre une tâche	33
2.11	Diagramme de séquences système — Voir les résultats	34
2.12	Diagramme de séquences système — Authentification	35
2.13	Diagramme de séquences système — Consulter une machine	36
2.14	Interaction entre le rôle de réception et son environnement	42
2.15	Interaction entre le réceptionniste et le générateur de sous-tâches	43
2.16	Interaction détaillée entre réception et génération des sous-calculs	43
2.17	Interaction entre le générateur de sous-calculs et l'orchestrateur	44
2.18	Interaction entre les agents d'exécution et le synthétiseur	45
2.19	Interaction entre le synthétiseur et le réceptionniste	46
2.20	Architecture Agentique	48
2.21	Workflow du processus de décomposition des calculs	58
2.22	Workflow de recomposition des calculs	60
2.23	Architecture à deux couches du heartbeat.	61
2.24	Machine à états du suivi d'un pair par un observateur.	62
2.25	Diagramme de classe metiers	64
2.26	parallax-AUTHENTIFICATION	65
2.27	consulter machines	65
2.28	consulter taches	66
2.29	CREATE ACCOUNT	66
2.30	Execution de tache	67
2.31	importer des taches	67

2.32 enlever un noeud	68
2.33 Diagramme d'état d'une tache	69
2.34 Diagramme d'état d'un programme	70
2.35 Diagramme d'état d'un noeud	71
2.36 Diagramme des composants - parallax	72

Liste des tableaux

1.1	Comparaison des solutions existantes	18
2.1	Caractéristiques des machines du cluster PARALLAX	23
2.2	Cas d'utilisation : Authentification	27
2.3	Cas d'utilisation : Création de compte	27
2.4	Cas d'utilisation : Importation d'une tâche	28
2.5	Cas d'utilisation : Soumission d'une tâche	28
2.6	Cas d'utilisation : Consultation des résultats	29
2.7	Cas d'utilisation : Consultation d'une machine	29
2.8	Cas d'utilisation : Suppression d'une machine du cluster	30
2.9	Cas d'utilisation : Consultation d'un programme en cours d'exécution	31
2.10	Table des actions du système SMA_PARALLAX	41
2.11	Table des événements du système SMA_PARALLAX	47
2.12	Table des annotations du système Parallax	54
2.13	Paramètres du protocole de heartbeat	61

Introduction Générale

Les chercheurs dans des domaines tels que les mathématiques, la biologie et la physique réalisent aujourd'hui des entraînements de modèles d'intelligence artificielle ainsi que des simulations complexes dans leurs disciplines respectives. Ces activités nécessitent une puissance de calcul très importante, généralement fournie par des infrastructures distantes telles que des serveurs en ligne ou des plateformes de calcul spécialisées.

En particulier, les chercheurs de l'ENSPY utilisent ces services en ligne, qui sont non seulement coûteux, mais également fortement dépendants de la qualité de la connexion Internet. Dans notre contexte, cette connexion est souvent lente, ce qui engendre des retards significatifs dans l'exécution des travaux. Pourtant, le campus dispose de plus de 200 machines fonctionnelles qui restent sous-exploitées.

Dès lors, une question se pose : comment fédérer les ressources matérielles existantes afin de mettre en place une infrastructure capable de réduire les coûts et de limiter les délais causés par les contraintes de connectivité, tout en offrant une solution structurée et réutilisable à travers une documentation claire destinée aux utilisateurs et aux développeurs ?

Hypothèse : la mise en place d'un supercalculateur pourrait constituer une solution à ce problème. Cependant, la réalisation d'un tel système soulève plusieurs interrogations quant à sa conception, son déploiement et surtout son utilisation, qui devra être guidée par une documentation technique permettant aux utilisateurs de programmer correctement leurs calculs sur l'infrastructure.

Le projet PARALLAX a pour objectifs : d'exploiter les machines existantes afin de construire un supercalculateur ; de permettre aux chercheurs d'exécuter leurs calculs sur cette infrastructure ; et de fournir une documentation complète décrivant les interfaces, les annotations et les mécanismes nécessaires pour programmer et soumettre efficacement des tâches sur le système.

Notre méthodologie se décline en deux axes : physique et logique. Sur le plan physique, il s'agit de récupérer les machines, de diagnostiquer leurs caractéristiques, de les réparer si nécessaire, puis de les interconnecter en réseau et de tester leur connectivité. Sur le plan logique, il s'agit de développer les programmes nécessaires, de définir une architecture logicielle cohérente et de mettre en place une documentation permettant aux utilisateurs de comprendre comment coder et exécuter leurs applications sur l'infrastructure.

Dans les chapitres suivants, nous présenterons d'abord les concepts nécessaires ainsi que l'état de l'art dans le chapitre 1. Le chapitre 2 sera consacré à l'analyse et à la conception

du projet. Enfin, le chapitre 3 abordera l'implémentation, les tests ainsi que la documentation technique destinée à l'utilisation du système.

Chapitre 1

Généralités et État de l’art

1.1 Généralités

Cette section présente les notions fondamentales nécessaires à la compréhension de ce travail, notamment celles liées au calcul distribué, au parallélisme et aux systèmes multi-agents. Ces concepts constituent les bases théoriques sur lesquelles repose le système développé.

1.1.1 Généralités

- **Calcul distribué** : Le calcul distribué désigne une approche informatique consistant à répartir l’exécution d’un calcul sur plusieurs machines interconnectées. L’objectif principal est d’améliorer les performances, la scalabilité et la tolérance aux pannes en exploitant la puissance de calcul collective d’un ensemble de nœuds [5].
- **Parallélisme** : Le parallélisme consiste à exécuter simultanément plusieurs opérations ou tâches afin de réduire le temps global d’exécution. Il peut être basé sur les données ou sur les instructions, selon la structure du problème à résoudre [5].
- **Agent** : Un agent est une entité logicielle autonome capable de percevoir son environnement, de prendre des décisions et d’agir pour atteindre des objectifs spécifiques [6].
- **Système multi-agents** : Un système multi-agents (SMA) est un ensemble d’agents interagissant entre eux dans un environnement partagé. Ces interactions permettent la coordination, la coopération et parfois la compétition afin de résoudre un problème global [6].
- **Orchestrateur** : L’orchestrateur est un composant chargé de la répartition et de la coordination des tâches entre les différents agents ou nœuds du système. Il optimise l’utilisation des ressources disponibles et assure une exécution efficace des calculs [5, 8].
- **Contrôleur** : Le contrôleur est un nœud spécialisé chargé de la supervision continue du cluster. Il centralise les informations envoyées périodiquement par les autres nœuds (heartbeat), maintient une vue globale et actualisée de leur état (ressources disponibles,

activité, connectivité) et analyse ces données afin de détecter rapidement toute anomalie ou défaillance [5].

Au-delà de la simple surveillance, le contrôleur joue un rôle clé dans la stabilité du système. Il déclenche les mécanismes de gestion des pannes, tels que la mise hors service d'un nœud défaillant ou l'initiation d'un processus d'élection pour remplacer un rôle critique [9]. Il garantit ainsi la cohérence, la fiabilité et la continuité du fonctionnement global du cluster.

- **Tâche** : Une tâche représente une unité de travail à exécuter dans le système. Dans un contexte distribué, une tâche peut être décomposée en sous-tâches afin d'être exécutée en parallèle sur plusieurs nœuds [7].
- **Remplaçant** : Le remplaçant est un nœud du cluster désigné pour assurer la continuité du système en cas de défaillance d'un rôle critique, notamment le maître ou le contrôleur. Il se maintient en état de veille active, avec une connaissance à jour de l'état global du cluster, afin de pouvoir reprendre immédiatement les responsabilités du nœud défaillant sans interruption significative du service.

Au-delà de ce rôle de secours, le remplaçant participe au mécanisme global de tolérance aux pannes en garantissant qu'aucune fonction essentielle du système ne reste vacante. Lorsqu'il prend le relais, un processus d'élection est automatiquement déclenché pour désigner un nouveau remplaçant, assurant ainsi la pérennité et la résilience de l'architecture distribuée [9].

- **Ordonnanceur** : L'ordonnanceur est responsable de la planification de l'exécution des tâches. Il décide de l'ordre et des ressources allouées à chaque tâche en fonction de critères tels que la charge ou la priorité [7].
- **DAG (Directed Acyclic Graph)** : Un DAG est un graphe orienté sans cycle représentant les dépendances entre tâches. Il est utilisé pour modéliser l'ordre d'exécution dans les systèmes parallèles et distribués [7].
- **Heartbeat** : Le heartbeat est un mécanisme de communication périodique par lequel chaque nœud du cluster transmet son état au contrôleur. À intervalles réguliers, il envoie un message contenant des informations telles que son identifiant, ses ressources disponibles (CPU, mémoire), ainsi que d'autres indicateurs de fonctionnement, permettant d'obtenir une vision actualisée de l'ensemble du système [5, 8].

Au-delà de la simple signalisation de présence, le heartbeat constitue un élément essentiel de la supervision. Il permet au contrôleur de détecter rapidement les défaillances en cas d'absence de message dans un délai donné, d'évaluer la charge des machines et de maintenir la cohérence globale du cluster en facilitant les décisions d'orchestration et de réallocation des tâches.

- **Broadcast** : Le broadcast est un mécanisme de communication consistant à diffuser un message à l'ensemble des nœuds d'un système. Il est souvent utilisé pour la synchronisation

ou la diffusion d'informations globales [5].

1.2 État de l'art

L'accès à des ressources de calcul à faible coût dans des environnements à connectivité limitée reste un défi majeur dans de nombreux contextes scientifiques et académiques. Plusieurs solutions concrètes ont été proposées pour répondre à ce problème. Ces solutions explorent différentes stratégies, notamment la mutualisation des ressources, la réduction des coûts ou encore l'adaptation aux contraintes réseau et aux environnements instables.

1.2.1 SETI@home : calcul distribué volontaire à grande échelle

SETI@home est un projet scientifique développé à l'Université de Californie à Berkeley [1]. Il vise à analyser des signaux radio provenant de l'espace en exploitant la puissance de calcul d'ordinateurs personnels répartis à travers le monde. L'objectif est de transformer une masse de machines hétérogènes en une infrastructure de calcul globale.

Chaque utilisateur installe un logiciel client qui télécharge des fragments de données, exécute les calculs localement, puis renvoie les résultats à un serveur central. Ce fonctionnement permet de découper une tâche globale en unités indépendantes exécutées en parallèle sur un très grand nombre de machines.

Cette architecture permet d'atteindre une puissance de calcul très importante sans infrastructure dédiée et sans investissement matériel centralisé. Elle réduit fortement les coûts et rend possibles des calculs scientifiques à grande échelle.

Cependant, le système repose entièrement sur une connectivité Internet stable et continue. Il nécessite une communication permanente pour distribuer les tâches et récupérer les résultats, ce qui introduit une forte dépendance au réseau.

1.2.2 AWS Snowball : adaptation aux environnements à faible connectivité

AWS Snowball est une solution proposée par Amazon Web Services[2] pour transférer de grandes quantités de données dans des environnements où la bande passante est insuffisante ou instable. Elle répond principalement aux limitations liées au transfert de données volumineuses vers le cloud.

L'utilisateur copie les données sur un dispositif physique sécurisé et chiffré. Ce dispositif est ensuite transporté vers un centre de données où les informations sont intégrées dans l'infrastructure cloud.

Cette approche permet de contourner les limitations liées aux connexions lentes ou coûteuses. Elle est particulièrement utilisée dans les contextes de migration de données massives.

Cependant, cette solution reste centrée sur le transfert de données et non sur l'exécution de calculs interactifs. Elle ne permet pas un accès continu aux ressources de calcul.

1.2.3 Amazon EC2 Spot Instances : réduction du coût du calcul

Amazon EC2 Spot Instances est un service proposé par Amazon Web [3] Services permettant d'accéder à des ressources de calcul à prix réduit. Ces ressources correspondent à des machines virtuelles disponibles temporairement dans les centres de données.

Les utilisateurs exploitent ces instances pour exécuter des charges de travail intensives telles que des simulations ou des traitements batch. Ce modèle permet de réduire significativement les coûts d'exécution.

Cette approche exploite les capacités inutilisées des infrastructures cloud pour proposer des tarifs dynamiques plus faibles.

Cependant, ces ressources peuvent être interrompues à tout moment. Leur utilisation nécessite également une connexion continue au cloud.

1.2.4 HTCondor : exploitation locale des ressources disponibles

HTCondor est un système de calcul distribué développé à l'Université du Wisconsin-Madison[4]. Il permet de transformer un ensemble de machines locales en une plateforme de calcul partagée.

Dans les environnements académiques, il est utilisé pour exploiter les machines lorsqu'elles sont inactives. Les tâches sont planifiées automatiquement selon la disponibilité des ressources.

Cette solution repose sur une exploitation opportuniste des ressources locales. Elle réduit les coûts et limite la dépendance au réseau Internet.

Cependant, elle nécessite une gestion avancée des ressources et un système d'ordonnancement efficace.

1.3 Bilan et positionnement

1.3.1 Bilan

Étude de l'existant

Les solutions existantes proposent différentes approches pour réduire le coût du calcul et améliorer la gestion des contraintes réseau. Certaines reposent sur la mutualisation massive des ressources distribuées, tandis que d'autres exploitent des ressources locales ou des modèles économiques flexibles dans le cloud.

Cependant, ces solutions présentent des limites importantes dans des environnements académiques à ressources contraintes. Les approches cloud restent fortement dépendantes d'une

connectivité Internet stable. Les solutions de calcul distribué à grande échelle nécessitent également une infrastructure globale et une coordination permanente. Enfin, les solutions locales sont souvent limitées à des environnements fortement structurés et difficiles à généraliser.

Afin de comparer ces approches, les critères suivants sont utilisés : réduction du coût, dépendance au réseau et exploitation des ressources locales.

Critères	SETI@home	AWS Snowball	EC2 Spot	HTCondor
Réduction du coût	✓	×	✓	✓
Faible dépendance réseau	×	✓	×	✓
Exploitation locale	×	×	×	✓

TABLE 1.1 – Comparaison des solutions existantes

Légende : ✓ : satisfait × : non satisfait

1.3.2 Positionnement

Ce projet se positionne comme une solution de calcul distribuée locale adaptée aux environnements académiques à ressources limitées.

Contrairement aux solutions cloud comme EC2 Spot Instances, il ne dépend pas d'une connexion Internet continue. Contrairement aux approches de calcul distribué global comme SETI@home, il n'exige pas une infrastructure mondiale ni des utilisateurs externes. Enfin, contrairement aux solutions de transfert physique comme AWS Snowball, il permet l'exécution directe des tâches de calcul.

L'approche proposée exploite des machines locales inutilisées afin de constituer un cluster de calcul distribué. Elle vise à réduire les coûts d'infrastructure tout en assurant une autonomie vis-à-vis de la connectivité Internet.

Cette solution est particulièrement adaptée aux environnements académiques où des ressources matérielles existent mais restent sous-utilisées.

Chapitre 2

Analyse et conception

2.1 Analyse

Face à la dépendance coûteuse des chercheurs de l'ENSPY envers des serveurs distants et aux limitations d'une connectivité Internet insuffisante, le projet PARALLAX se propose de fédérer les ressources des machines sous-utilisées du campus en une infrastructure de calcul distribué locale. Pour mener à bien cette solution, nous avons procédé à une analyse selon deux axes : d'une part l'axe matériel, qui dresse l'inventaire des machines disponibles pour constituer le cluster, et d'autre part l'axe logiciel, qui permet l'identification des acteurs du système et de leurs besoins fonctionnels.

2.1.1 Analyse matérielle du cluster PARALLAX

Les machines DELL

Le cluster dispose de trois ordinateurs de bureau DELL. Deux d'entre eux sont des Dell OptiPlex 380 et le troisième est un Dell System 360.

Dell OptiPlex 380 — Machine 1

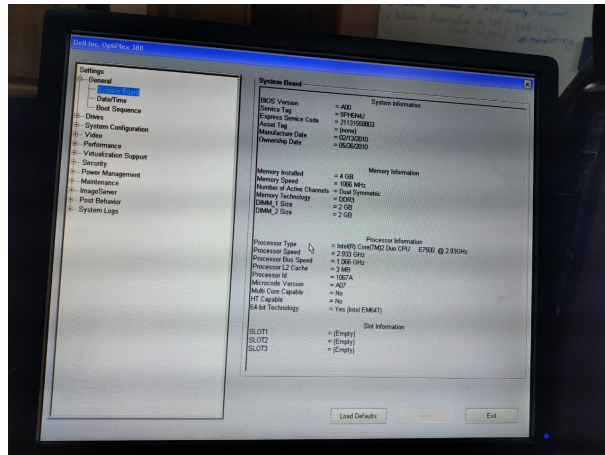


FIGURE 2.1 – Dell OptiPlex 380 — Machine 1

Le processeur est un Intel Core 2 Duo E7500 cadencé à 2,933 GHz, doté de 2 cœurs physiques et d'un cache L2 de 3 MB. La mémoire RAM est de 4 GB de type DDR3 à 1066 MHz, répartie en deux barrettes de 2 GB chacune en mode Dual Symmetric. La machine supporte la technologie 64 bits.

Dell OptiPlex 380 — Machine 2

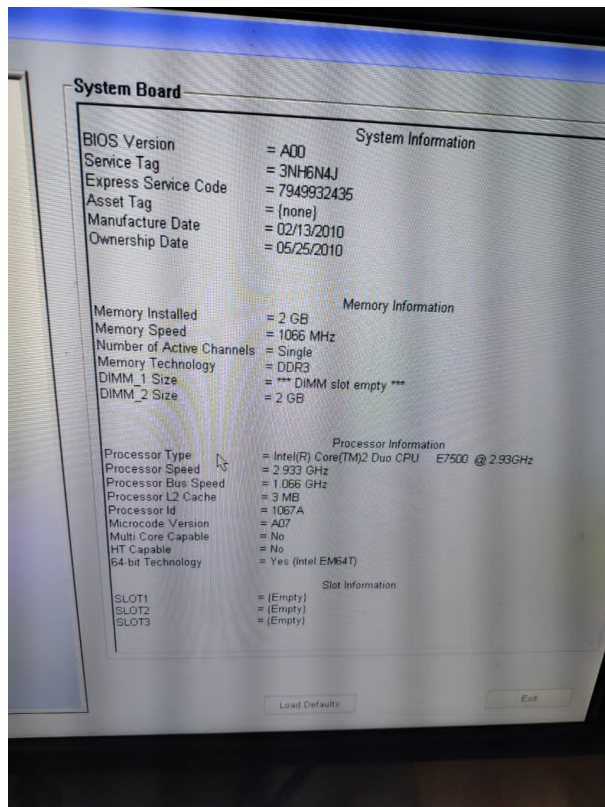


FIGURE 2.2 – Dell OptiPlex 380 — Machine 2

Le processeur est identique au premier : Intel Core 2 Duo E7500 à 2,933 GHz, 2 cœurs, cache L2 de 3 MB. La mémoire RAM est de 2 GB de type DDR3 à 1066 MHz, mais un seul slot

DIMM est occupé, le premier slot étant vide. La machine fonctionne en mode Single Channel, ce qui réduit les performances mémoire par rapport à la première machine.

Dell System 360 — Machine 3

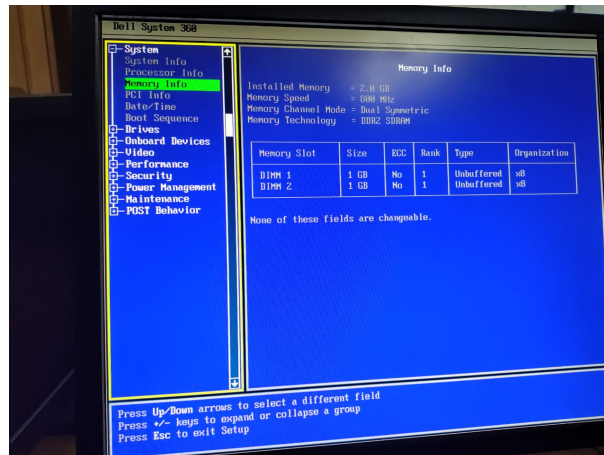


FIGURE 2.3 – Dell System 360 — Machine 3

La mémoire RAM est de 2 GB de type DDR2 à 800 MHz, répartie en deux barrettes de 1 GB chacune en mode Dual Symmetric. La technologie mémoire DDR2 est une génération en dessous des deux OptiPlex 380, ce qui implique une bande passante mémoire inférieure.

Les mini-ordinateurs

Le cluster intègre également trois thin clients de type Wyse. Deux d'entre eux sont équipés d'un processeur à 533 MHz, de 32 MB de RAM et de 32 MB de stockage flash. Le troisième, de gamme supérieure, dispose d'un processeur à 1 GHz, de 64 MB de RAM et de 64 MB de stockage flash. Ces trois appareils fonctionnent sous Windows CE et sont structurellement différents des postes DELL : leur faible capacité de traitement les exclut du rôle de coordinateur et les cantonne à des tâches légères de calcul au sein du cluster.

Tiny Client ×2 (533 MHz)

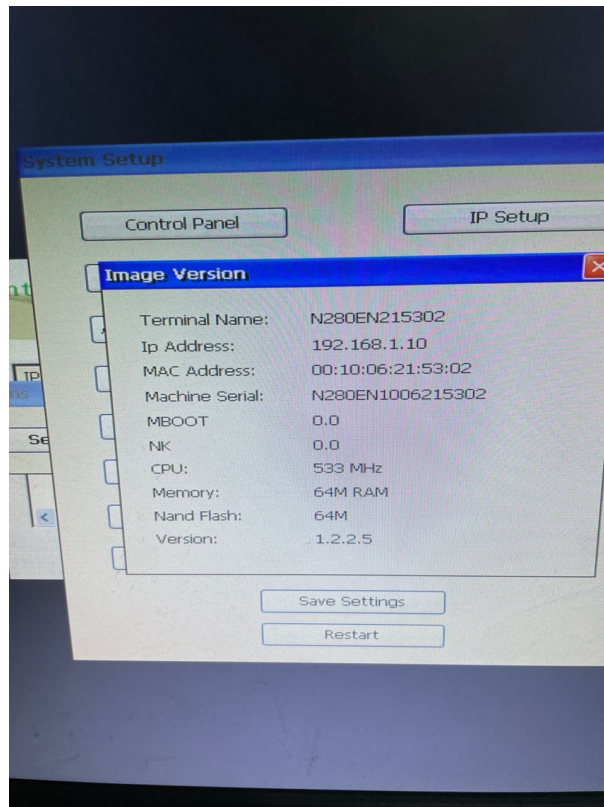


FIGURE 2.4 – Thin client Wyse — 533 MHz (×2)

Tiny Client Wyse (1 GHz)

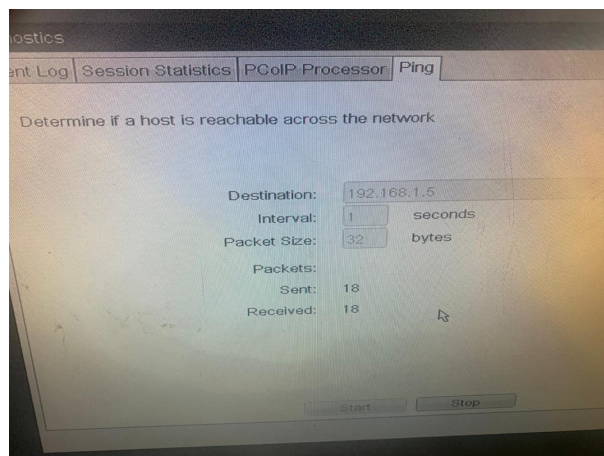


FIGURE 2.5 – Thin client Wyse — 1 GHz

Infrastructure réseau

Les machines sont interconnectées via un switch Ethernet à 5 ports. Six câbles réseau droits RJ45 ont été sertis manuellement par l'équipe pour les besoins du projet. Les tests de connectivité réalisés via des commandes ping entre toutes les machines ont été concluants, validant ainsi la couche physique du réseau sur laquelle repose l'ensemble de l'architecture logicielle de

PARALLAX.



SPARK 30C

27mm f/1.6 1/40s ISO870

FIGURE 2.6 – Switch Ethernet 5 ports et câbles RJ45 sertis manuellement

Récapitulatif du matériel

Le tableau ci-dessous résume les caractéristiques de l'ensemble des machines constituant le cluster :

Caractéristique	DELL OptiPlex 380 — M1	DELL OptiPlex 380 — M2	DELL System 360 — M3	Tiny Client ×2	Tiny Client Wyse
Processeur	Intel Core 2 Duo E7500	Intel Core 2 Duo E7500	Non visible	—	—
Fréquence	2,933 GHz	2,933 GHz	—	533 MHz	1 GHz
Nombre de cœurs	2	2	—	—	—
Cache L2	3 MB	3 MB	—	—	—
RAM totale	4 GB	2 GB	2 GB	32 MB	64 MB
Type de RAM	DDR3 1066 MHz	DDR3 1066 MHz	DDR2 800 MHz	—	—
Mode mémoire	Dual Symmetric	Single Channel	Dual Symmetric	—	—
Stockage	—	—	—	32 MB flash	64 MB flash
Architecture	64 bits	64 bits	—	—	—
Système d'exploitation	Ubuntu Server 18.04	Ubuntu Server 18.04	Ubuntu Server 18.04	Windows CE	Windows CE
Rôle dans le cluster	Maître / Worker	Maître / Worker	Maître / Worker	Worker léger	Worker léger

TABLE 2.1 – Caractéristiques des machines du cluster PARALLAX

Limites matérielles par rapport au projet

Hétérogénéité des systèmes d'exploitation. Les trois DELL fonctionnent sous Ubuntu Server 18.04 tandis que les trois thin clients fonctionnent sous Windows CE. Cette différence impose à PARALLAX de gérer une couche de sérialisation des données assurant l'interopérabilité entre les deux systèmes.

Puissance de calcul limitée des DELL. Les processeurs Intel Core 2 Duo datent de 2010 et ne disposent que de 2 cœurs physiques chacun. Le bénéfice du parallélisme sera donc limité par la faible puissance individuelle de chaque nœud.

Hétérogénéité des DELL entre elles. La première machine dispose de 4 GB de RAM en Dual Channel, la deuxième de 2 GB en Single Channel, et la troisième de 2 GB de DDR2 moins rapide. Cette hétérogénéité justifie le recours au score pondéré dans l'algorithme de vote pour l'élection du nœud remplaçant.

Capacité très limitée des thin clients. Avec 32 à 64 MB de RAM et des processeurs entre 533 MHz et 1 GHz, ces machines sont donc défavorisées dans le processus d'élection du remplaçant et participent au cluster uniquement pour des tâches légères.

Contrainte du switch à 5 ports. Cette contrainte fixe le nombre maximum de nœuds actifs dans cette première phase du projet. Toute extension nécessiterait le remplacement du switch par un modèle de plus grande capacité.

Absence de redondance réseau. Le switch constitue un point de défaillance unique au niveau de la couche physique. Sa défaillance entraînerait la perte totale de connectivité du cluster, indépendamment de la robustesse de la couche logicielle.

2.1.2 Analyse logicielle

Synthèse des intervenants et fonctionnalités

Le système PARALLAX implique deux catégories d'acteurs.

1. e **Chercheur** (étudiant, enseignant ou chercheur de l'ENSPY) soumet ses programmes annotés via une interface CLI et consulte les résultats d'exécution.
2. . Le **Gestionnaire de cluster** administre l'infrastructure via une interface Web : il supervise l'état des nœuds grâce aux heartbeats, et gère l'ajout ou le retrait de machines du cluster.

Diagramme de contexte

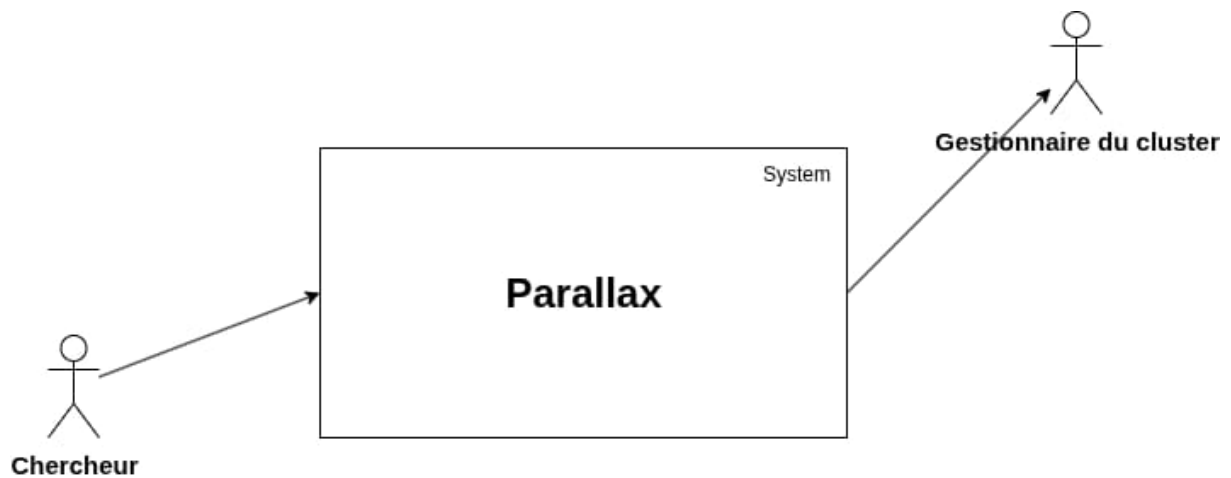


FIGURE 2.7 – Diagramme de Contexte

Fonctionnalités et exigences

Exigences fonctionnelles et non fonctionnelles

Exigences fonctionnelles :

- Soumission et exécution distribuée de tâches de calcul (Chercheur).

Exigences non fonctionnelles :

Présentation des cas d'utilisation

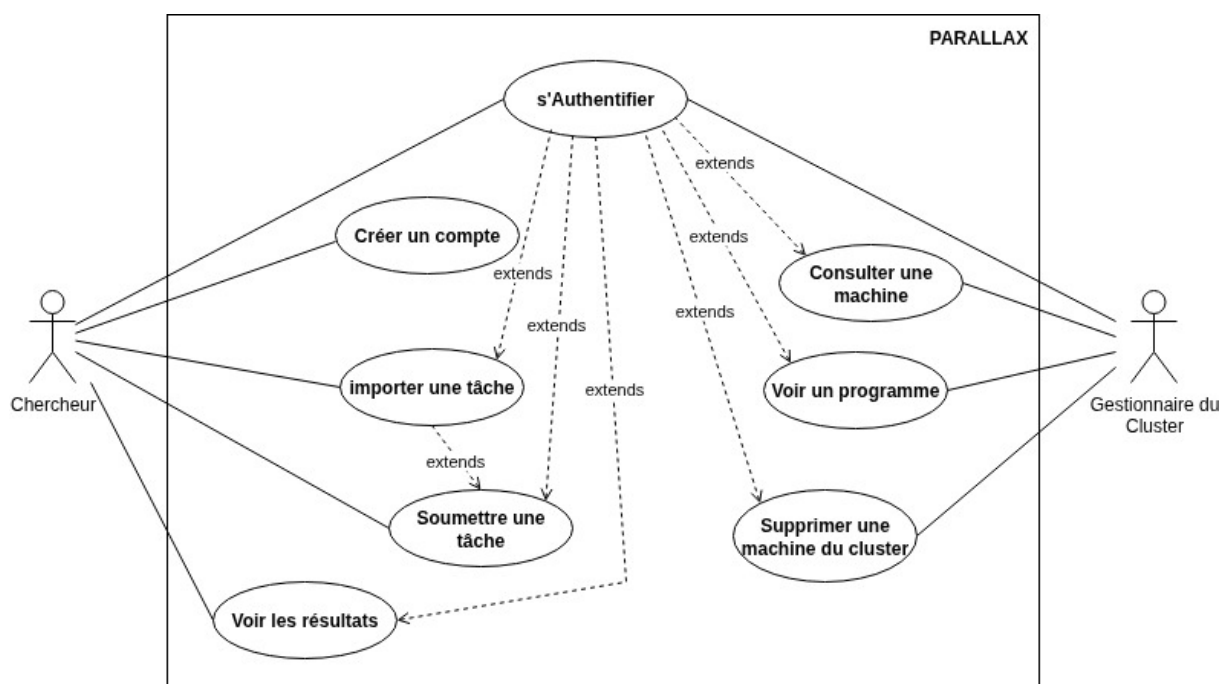


FIGURE 2.8 – Diagramme de cas d'utilisation

Objectif	Accéder aux fonctionnalités du système
Acteur(s)	Chercheur, Gestionnaire du cluster
Pré-condition	Aucune
Scénario nominal	1. L'acteur entre ses informations d'identification dans le système. 2. Le système vérifie l'exactitude des informations. 3. Les informations sont correctes, le système accorde l'accès aux fonctionnalités correspondantes au statut de l'utilisateur.
Scénarios alternatifs	— A1 : Erreur de saisie — Le système informe l'utilisateur et demande de nouvelles informations. — A2 : Erreur de connexion à la base de données — Le système renvoie un message d'erreur.
Post-condition	Accès aux fonctionnalités liées au statut de l'utilisateur

TABLE 2.2 – Cas d'utilisation : Authentification

Objectif	Créer un compte utilisateur
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'acteur entre ses informations dans le formulaire et le soumet au système. 2. Le système vérifie l'exactitude des informations. 3. Le système sauvegarde les informations et renvoie un message de succès.
Scénarios alternatifs	— A1 : Erreur de saisie — Le système demande à l'utilisateur de ressaisir ses informations. — A2 : Erreur de connexion à la base de données — Le système renvoie un message d'erreur.
Post-condition	Création du compte utilisateur

TABLE 2.3 – Cas d'utilisation : Création de compte

Objectif	Importer une tâche annotée
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur clique sur l'option d'importation d'une tâche. 2. Le système lui permet de sélectionner le dossier ou fichier annoté via un sélecteur de ressources. 3. L'utilisateur sélectionne et valide. 4. La tâche est enregistrée dans le système.
Scénarios alternatifs	— A1 : Document invalide — Le système annule l'importation et renvoie un message d'erreur.
Post-condition	Dossier / fichier importé dans le système

TABLE 2.4 – Cas d'utilisation : Importation d'une tâche

Objectif	Soumettre une tâche pour son exécution
Acteur(s)	Chercheur
Pré-condition	La tâche a été importée et le chercheur est authentifié
Scénario nominal	1. L'utilisateur sélectionne la tâche à soumettre et valide. 2. Le Master, grâce aux annotations, découpe et distribue la tâche à exécuter. 3. Les Workers exécutent la tâche et renvoient les résultats au Master pour recomposition. 4. Le Master recompose et renvoie le résultat à l'utilisateur.
Scénarios alternatifs	— A1 : Erreur de connexion — Un message d'erreur de connexion est renvoyé à l'utilisateur. — A2 : Dossier/fichier mal annoté — Le système renvoie un message d'erreur d'annotation.
Post-condition	Le résultat de l'exécution est renvoyé à l'utilisateur

TABLE 2.5 – Cas d'utilisation : Soumission d'une tâche

Objectif	Permettre à l'utilisateur de consulter les résultats de ses tâches exécutées
Acteur(s)	Chercheur
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne le menu des résultats. 2. Le système affiche les résultats et les détails d'exécution (durée, date, etc.).
Scénarios alternatifs	— A1 : Aucun résultat — Le système affiche un message indiquant l'absence de résultats. — A2 : Erreur de récupération — Un message d'erreur est renvoyé à l'utilisateur.
Post-condition	Les résultats sont affichés à l'utilisateur

TABLE 2.6 – Cas d'utilisation : Consultation des résultats

Objectif	Consulter l'état d'une machine du cluster
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne l'option de consultation des machines. 2. Le système affiche la liste des machines du cluster. 3. L'utilisateur sélectionne la machine souhaitée. 4. Le système affiche les détails de la machine sélectionnée.
Scénarios alternatifs	— A1 : Machine indisponible — Le système affiche un message d'erreur d'accessibilité. — A2 : Liste vide — Le système indique qu'aucune machine n'est disponible.
Post-condition	Les détails de la machine sont affichés

TABLE 2.7 – Cas d'utilisation : Consultation d'une machine

Objectif	Déconnecter une machine du cluster
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne la machine à arrêter dans la liste. 2. L'utilisateur déclenche la commande d'arrêt distant. 3. Le système demande à l'agent sur la machine d'exécuter la commande d'arrêt. 4. L'état de la machine est mis à jour sur l'interface pour confirmer l'arrêt.
Scénarios alternatifs	— A1 : Échec de communication — Le système affiche un message d'échec et propose de réessayer. — A2 : Refus de confirmation — L'utilisateur annule, la procédure est abandonnée sans modification. — A3 : Machine en cours d'exécution critique — Le système demande une confirmation supplémentaire ou bloque l'action.
Post-condition	La machine est arrêtée (déconnectée du cluster)

TABLE 2.8 – Cas d'utilisation : Suppression d'une machine du cluster

Objectif	Voir en temps réel l'état d'un programme en cours d'exécution
Acteur(s)	Gestionnaire du cluster
Pré-condition	Être authentifié
Scénario nominal	1. L'utilisateur sélectionne l'option de consultation des tâches en cours. 2. Le système affiche la liste des tâches actives. 3. L'utilisateur sélectionne la tâche souhaitée. 4. Le système affiche les détails d'exécution (machines impliquées, pourcentage d'avancement, etc.).
Scénarios alternatifs	— A1 : Aucune tâche en cours — Le système affiche un message d'absence de tâches actives. — A2 : Erreur de récupération — Le système affiche un message d'erreur et propose de réessayer.
Post-condition	Les détails d'exécution de la tâche sont affichés

TABLE 2.9 – Cas d'utilisation : Consultation d'un programme en cours d'exécution

Interaction Systemes

Presentation de Sequences Systeme

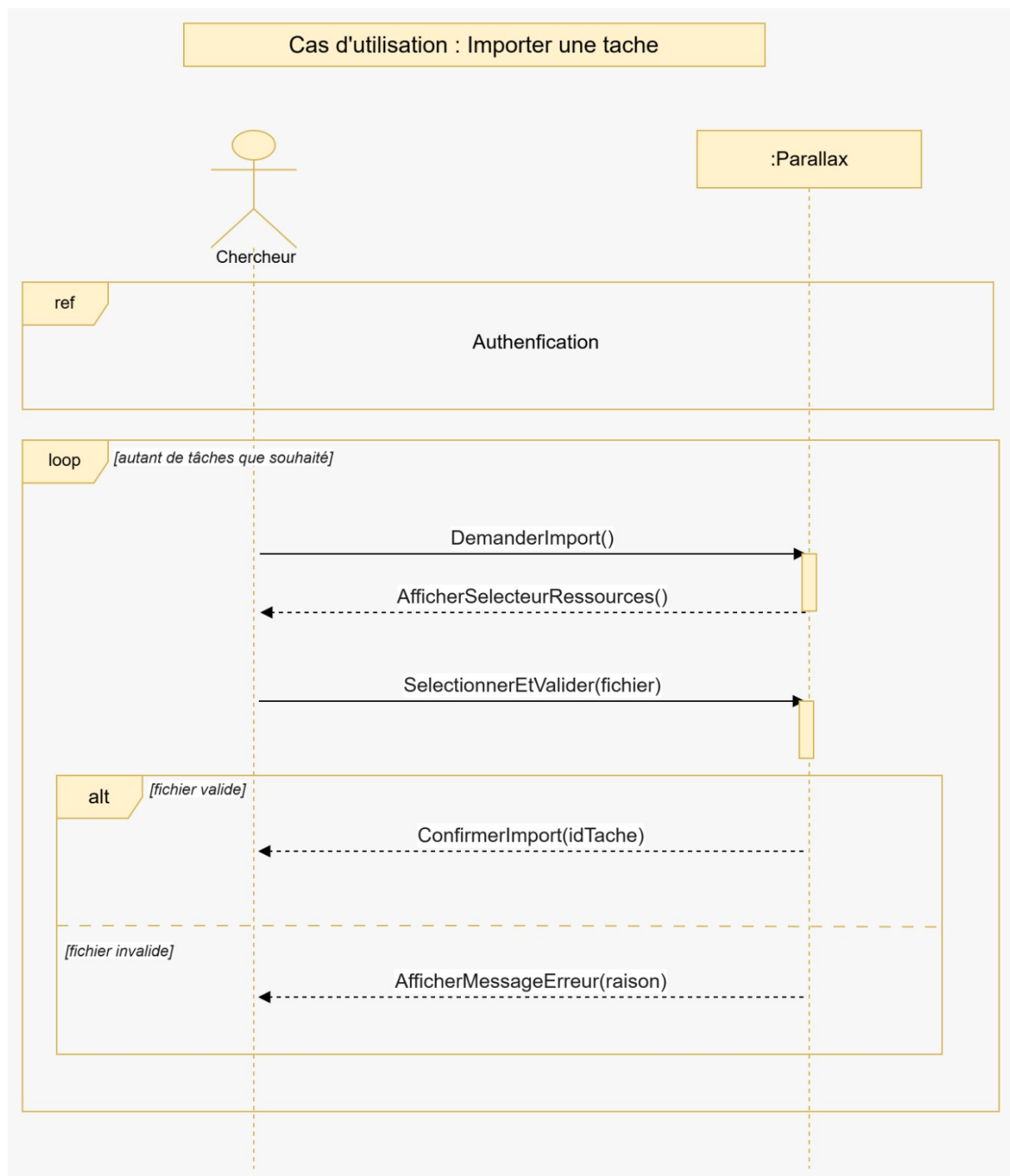


FIGURE 2.9 – Diagramme de séquences système — Importer une tâche

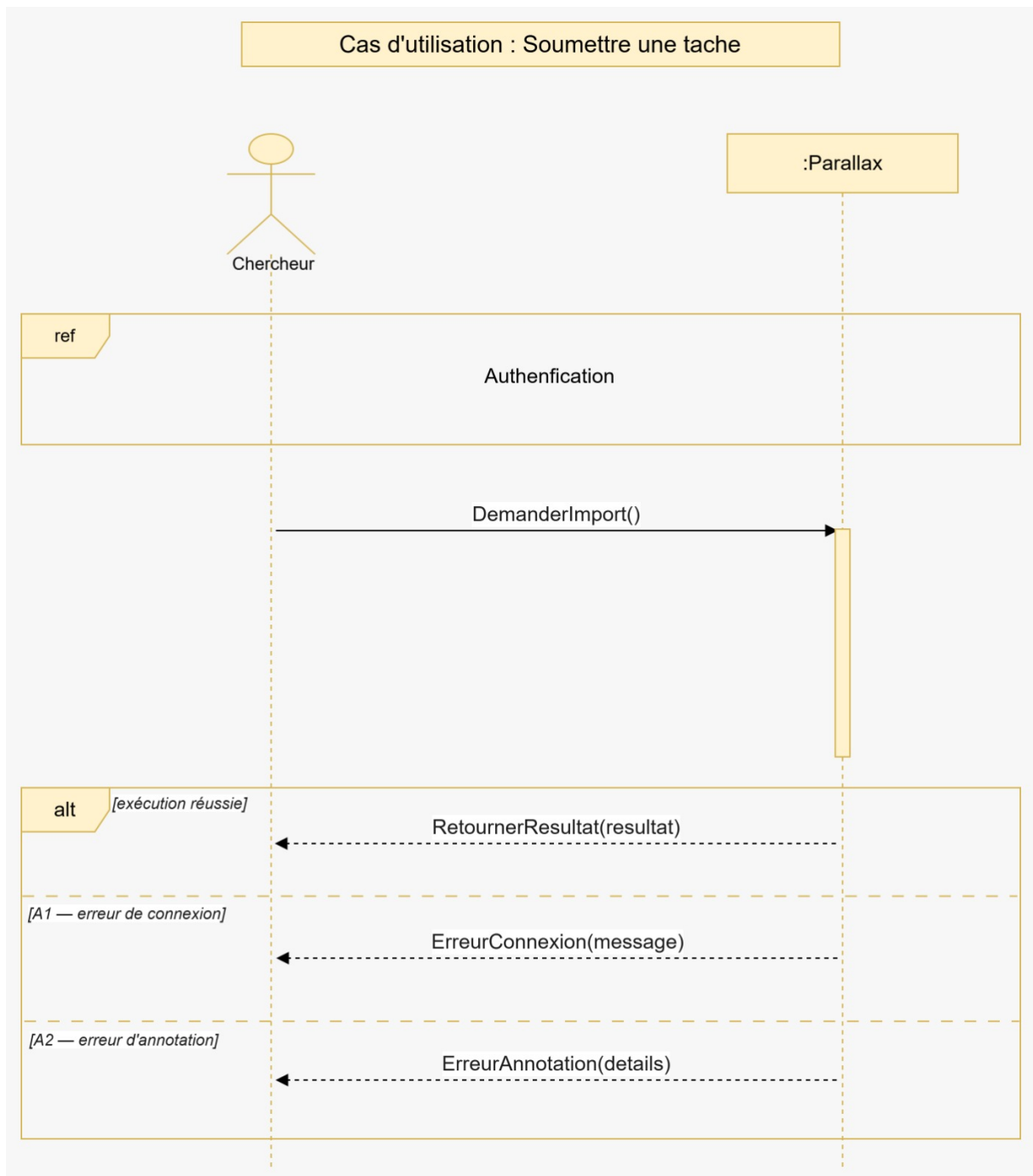


FIGURE 2.10 – Diagramme de séquences système — Soumettre une tâche

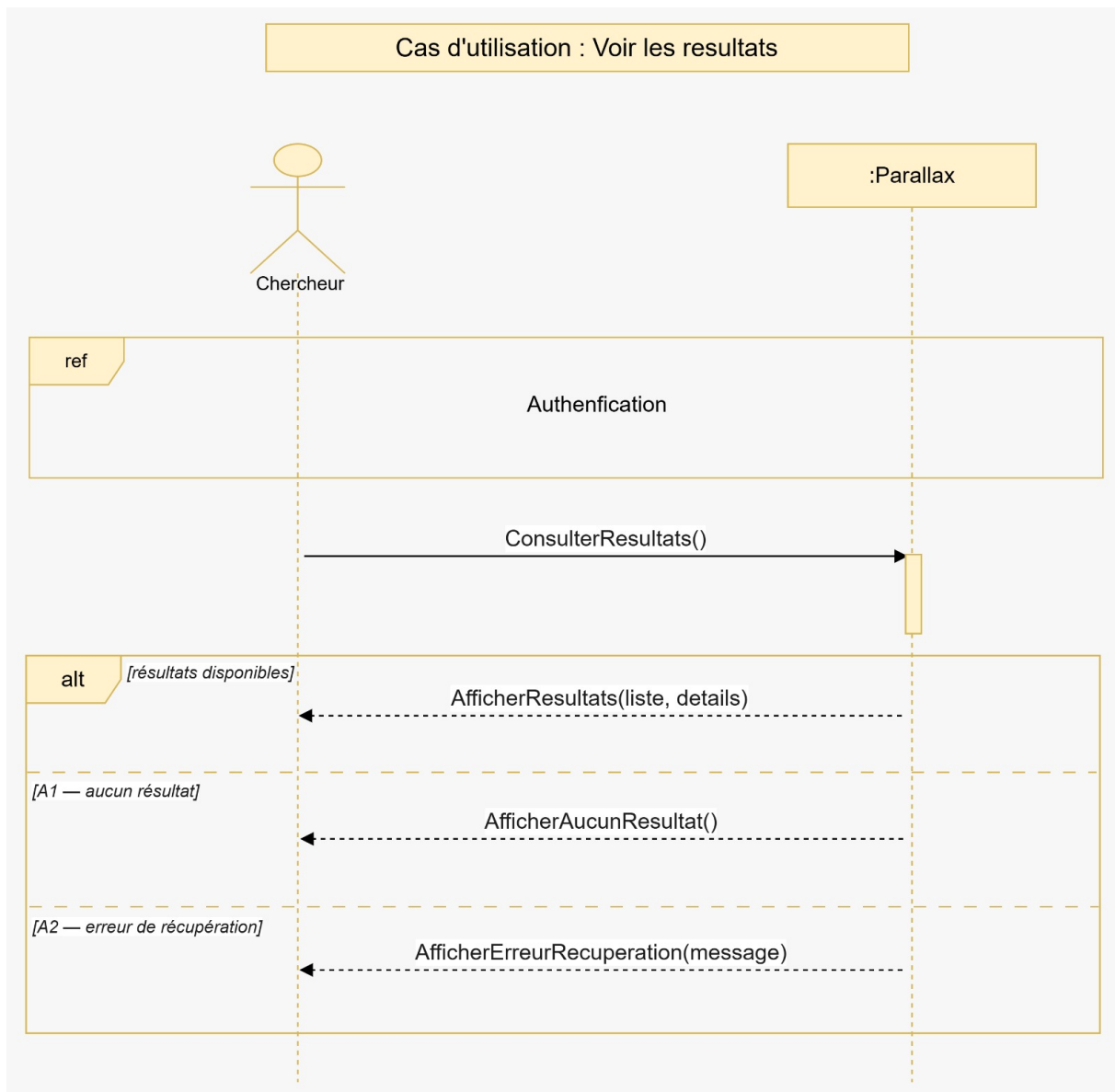


FIGURE 2.11 – Diagramme de séquences système — Voir les résultats

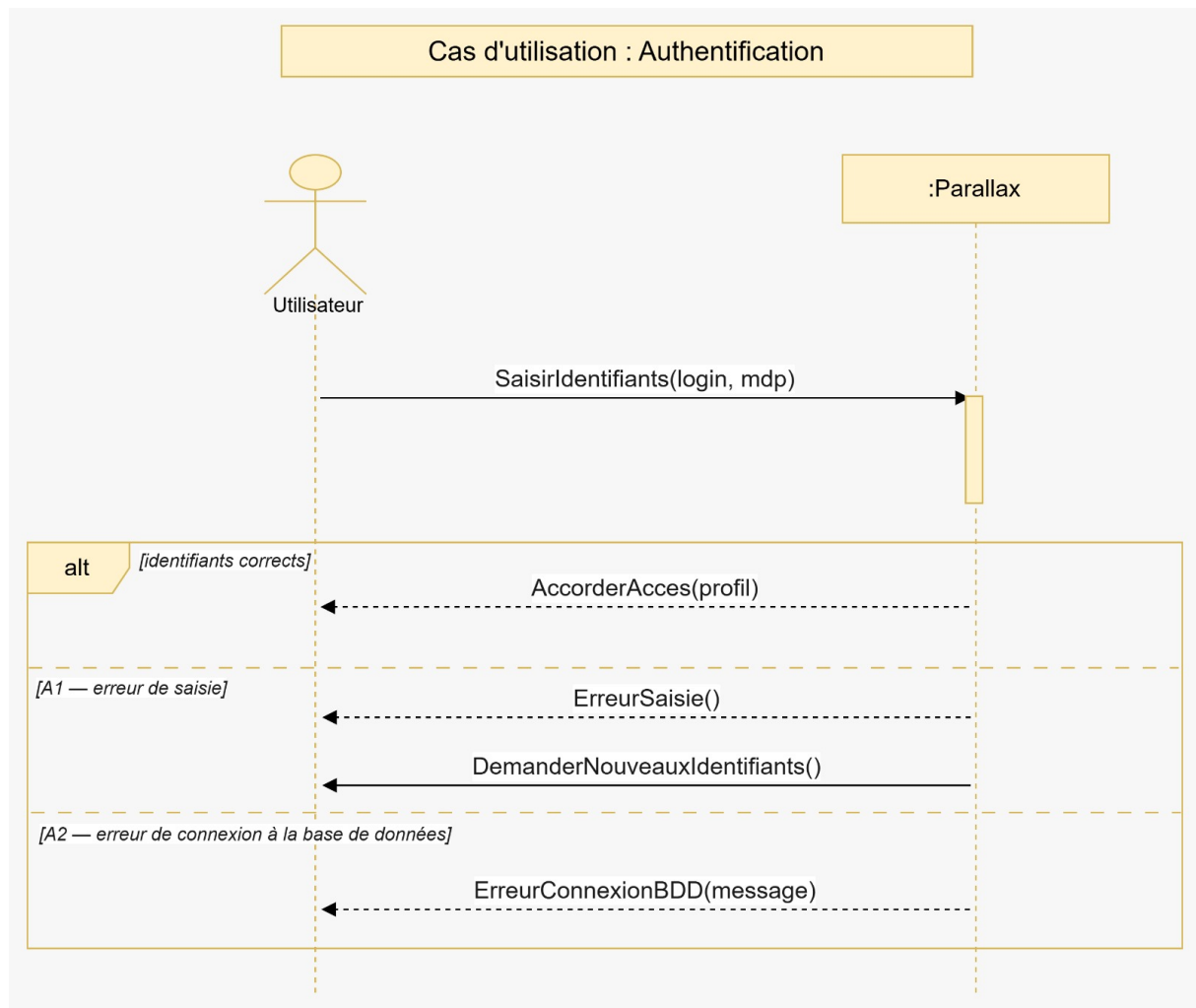


FIGURE 2.12 – Diagramme de séquences système — Authentification

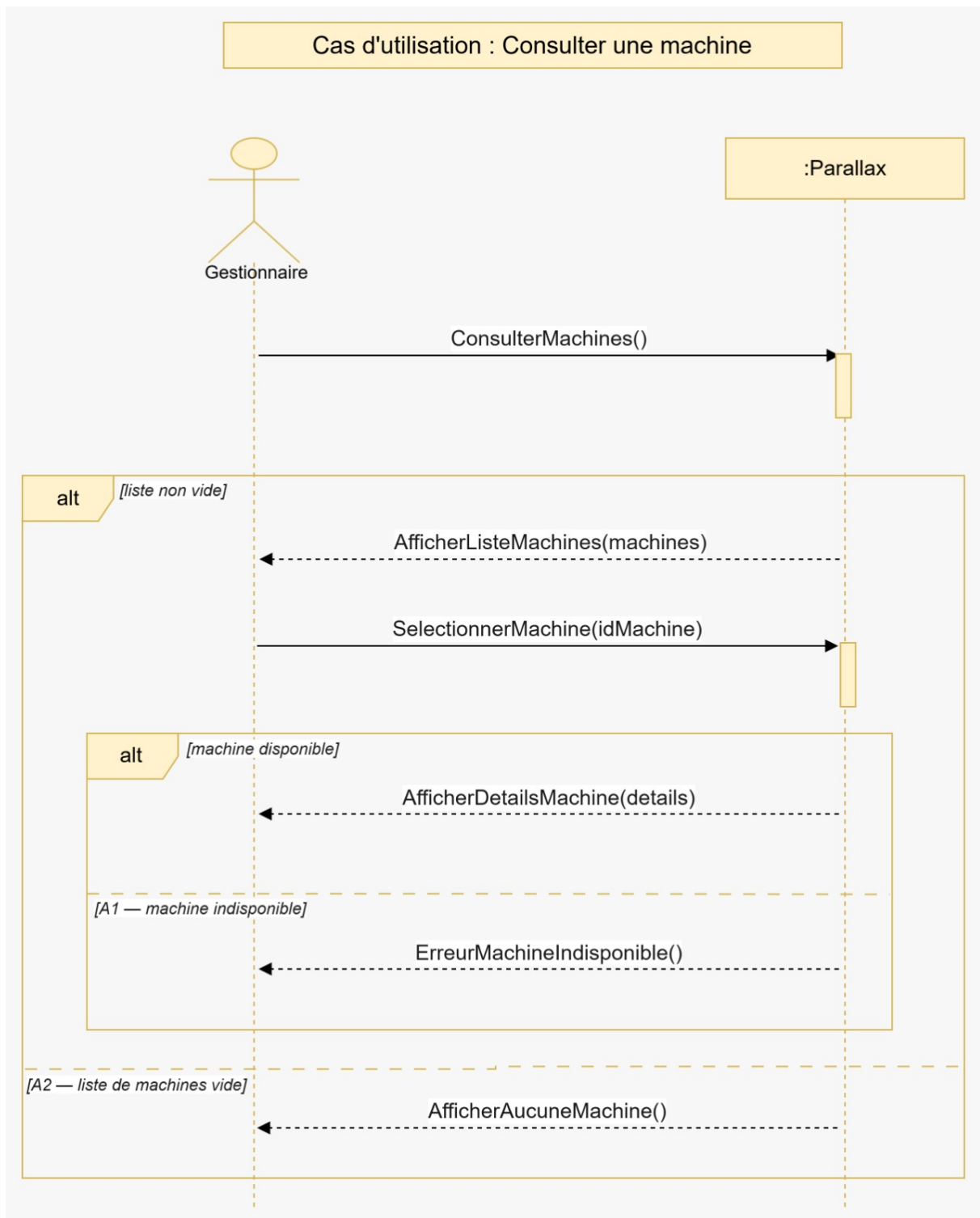


FIGURE 2.13 – Diagramme de séquences système — Consulter une machine

Presentations des Classes Metiers

Presentation des Etats Transition

2.2 Conception

Notre système permet aux chercheurs d'effectuer des calculs de simulation en s'appuyant sur des machines récupérées au sein de l'ENSPY. Ces machines sont mises en réseau et combinées afin de résoudre efficacement les problèmes soumis par les chercheurs.

Le système offre la possibilité aux utilisateurs de soumettre leurs calculs à partir d'une machine dédiée. Les tâches sont ensuite découpées en sous-calculs et distribuées aux différentes machines du réseau. Chaque machine exécute sa partie du calcul, puis renvoie les résultats à une machine centrale. Celle-ci se charge de les agréger afin de produire le résultat final.

Dans cette approche, chaque machine est considérée comme un **agent autonome**, ce qui permet de modéliser le système comme un **système multi-agents**.

2.2.1 Modélisation agentique du système

Vue globale du SMA Parrallax

Le fonctionnement global du système multi-agents Parrallax peut être résumé comme suit :

1. Réception du calcul soumis par le chercheur ;
2. Décomposition du calcul en sous-calculs ;
3. Répartition des sous-calculs entre les agents ;
4. Exécution des sous-calculs par les agents ;
5. Retour des résultats partiels ;
6. Recombinaison des résultats ;
7. Production du résultat final.

Modélisation d'un agent

Aspect physique Chaque machine $v \in V_M$ est caractérisée par un vecteur d'état décrivant ses différentes propriétés :

$$X_v = \begin{bmatrix} ID_v \\ NET_v \\ COMPUTE_v \\ MEMORY_v \\ STORAGE_v \\ LOAD_v \\ RELIABILITY_v \\ STATE_v \\ SECURITY_v \end{bmatrix}$$

Les composantes de ce vecteur représentent les différentes dimensions du système :

- * ID_v = uuid, role, region :
 - > *uuid* identifie de manière unique la machine ;
 - > *role* spécifie son rôle dans le système ;
 - > *region* indique sa localisation.
- * NET_v = ip, latency, bandwidth_{in}, bandwidth_{out}, jitter, loss : paramètres caractérisant les performances réseau.
- * $COMPUTE_v$ = cpucore, cpufreq, gpu, flops : capacités de calcul de la machine.
- * $MEMORY_v$ = ram, mem_bandwidth : ressources mémoire disponibles.
- * $STORAGE_v$ = disk, io : capacités de stockage et performances d'entrée/sortie.
- * $LOAD_v$ = cpuusage, ramusage, queue : niveau de charge courant de la machine.
- * $STATE_v$ = status, availability : état opérationnel de la machine.
- * $RELIABILITY_v$ = uptime, failurerate : indicateurs de fiabilité.
- * $SECURITY_v$ = auth, trust : paramètres liés à la sécurité et à la confiance.

Aspect logique Le système est organisé autour de différents rôles fonctionnels. Un agent peut endosser un ou plusieurs de ces rôles selon les besoins du système. Ces rôles définissent les responsabilités et les interactions attendues, sans correspondre nécessairement à des entités distinctes.

Rôles des agents

Les rôles fonctionnels au sein du système sont les suivants :

1. **Rôle de réception** : correspond à la prise en charge des calculs soumis via une interface utilisateur. Ce rôle implique l'entrée des requêtes dans le système.

2. **Rôle de contrôle réseau** : assure la surveillance de l'état global du système et la collecte des informations de statut des composants.
3. **Rôle de génération de sous-calculs** : consiste à décomposer un calcul global en unités plus petites et exploitables.
4. **Rôle d'orchestration** : implique la répartition des sous-calculs en fonction des ressources disponibles et de l'état du système.
5. **Rôle de synthèse** : consiste à agréger et recombinaer les résultats partiels afin de produire un résultat final cohérent.
6. **Rôle d'exécution** : correspond à la réalisation effective des calculs ou sous-calculs assignés.

Définition des agents du monde SMA_PARALLAX

Dans cette partie, nous décrivons les différents types d'agents du système SMA_PARALLAX ainsi que leurs caractéristiques et comportements agentiques.

1. Agent Master

L'agent Master est responsable de la réception, de la gestion et de la coordination de l'exécution des tâches. Il constitue le point central du système.

- **Environnement** : Interface utilisateur et système distribué.
- **Actions** :
 - **Réception des calculs** : Le nœud maître reçoit les requêtes de calcul soumises par les chercheurs via une interface.
 - **Décomposition des tâches** : Il décompose un calcul global en plusieurs sous-calculs.
 - **Réception des états du cluster** : Il reçoit, via le contrôleur, les informations d'état des différentes machines (CPU, RAM, latence) afin d'avoir une vision globale actualisée du cluster.
 - **Distribution des sous-calculs** : Il répartit les sous-calculs entre les agents Worker disponibles en tenant compte de leurs capacités et de leur état.
 - **Réception des résultats** : Il reçoit les résultats partiels provenant des Workers.
 - **Agrégation des résultats** : Il combine les résultats des sous-calculs afin de produire le résultat final.
 - **Exécution locale (optionnelle)** : Il peut exécuter certaines tâches légères localement.
 - **Observation du système** : Il surveille l'état des autres machines et agents.

- **Diffusion de son état** : Il communique son état au reste du système lorsque nécessaire.

2. Agent Worker

L'agent Worker est responsable de l'exécution des sous-tâches qui lui sont assignées. Il agit comme unité de calcul dans le système distribué.

- **Environnement** : Réseau distribué de machines interconnectées.
- **Actions** :
 - **Échange de messages** : Les Workers peuvent communiquer avec les autres nœuds du système.
 - **Réception des sous-calculs** : Ils reçoivent les tâches envoyées par l'agent Master.
 - **Exécution des sous-calculs** : Ils exécutent localement les tâches assignées.
 - **Retour des résultats** : Ils renvoient les résultats au Master après exécution.
 - **Diffusion de l'état** : Ils envoient périodiquement leur état (charge, disponibilité, performance).

3. Agent Contrôleur

L'agent Contrôleur est chargé de maintenir une vue globale cohérente de l'état du système, notamment en ce qui concerne la présence, la disponibilité et les caractéristiques des agents.

- **Environnement** : Ensemble des agents du réseau distribué.
- **Actions** :
 - **Détection des problèmes** : Il notifie l'agent Master en cas de panne ou de défaillance d'un nœud. En cas de défaillance du Master lui-même, il informe également les autres nœuds du cluster afin de déclencher un mécanisme approprié (où celui du vote est compris) et assurer la continuité du système.
 - **Réception des états** : Il collecte les informations d'état des différentes machines.
 - **Enregistrement des nœuds** : Il intègre les nouveaux nœuds dans le système.

N°	Nom de l'action	Commentaire	Rôle d'agent	Données d'entrée	Données de sortie
1	Réception des calculs	Prise en charge des requêtes utilisateur	Réception (Master)	Requête utilisateur	Tâche globale
2	Décomposition des tâches	Division du calcul global en sous-calculs	Génération (Master)	Tâche globale	Sous-calculs
3	Distribution des sous-calculs	Allocation des tâches aux Workers	Orchestration (Master)	Sous-calculs + état des Workers	Tâches assignées
4	Réception des sous-calculs	Réception des tâches envoyées par le Master	Exécution (Worker)	Sous-calculs	Tâches locales
5	Exécution des sous-calculs	Calcul effectif des tâches	Exécution (Worker)	Tâches locales	Résultats partiels
6	Retour des résultats	Envoi des résultats vers le Master	Exécution (Worker)	Résultats partiels	Résultats transmis
7	Agrégation des résultats	Combinaison des résultats partiels	Synthèse (Master)	Résultats partiels	Résultat final
8	Observation du système	Surveillance des ressources et états	Contrôle réseau (Master)	États des agents	Vue système
9	Diffusion de l'état Worker	Communication de la charge et disponibilité	Contrôle réseau (Worker)	État local	Infos d'état
10	Réception des états	Collecte des infos système	Contrôle réseau (Contrôleur)	Infos d'état	Base de données état
11	Détection des problèmes	Identification des pannes	Contrôle réseau (Contrôleur)	États système	Alertes
12	Notification des défaillances	Alerte envoyée au Master	Contrôle réseau (Contrôleur)	Alertes	Notification
13	Enregistrement des nœuds	Ajout de nouveaux agents	Contrôle réseau (Contrôleur)	Infos nouveau nœud	Nœud enregistré

TABLE 2.10 – Table des actions du système SMA_PARALLAX

2.2.2 Interactions entre agents

Dans un système multi-agents distribué tel que **SMA_PARALLAX**, les interactions entre agents constituent un élément central assurant la coordination, la cohérence et l'efficacité globale du système.

Ces interactions reposent principalement sur des mécanismes d'échange de messages et de partage d'informations d'état, permettant aux différents agents d'adapter dynamiquement leur comportement en fonction du contexte d'exécution. Chaque rôle fonctionnel (réception, génération, orchestration, exécution, synthèse et contrôle) n'agit pas de manière isolée, mais s'inscrit dans une chaîne d'interactions structurée formant un pipeline de traitement distribué.

Ainsi, le flux global du système peut être décrit comme suit :

- Une requête utilisateur est introduite dans le système via l'agent de réception.
- Cette requête est transformée en sous-tâches par l'agent générateur.
- Les sous-tâches sont ensuite distribuées de manière optimale par l'orchestrateur.
- Les agents d'exécution réalisent les calculs et produisent des résultats partiels.
- Enfin, l'agent de synthèse agrège ces résultats pour produire une sortie finale cohérente.

Parallèlement à ce flux principal, des interactions transversales assurent la supervision du système, notamment via la collecte et la diffusion d'informations d'état entre agents.

Les figures suivantes illustrent les principales interactions entre les différents rôles du système :

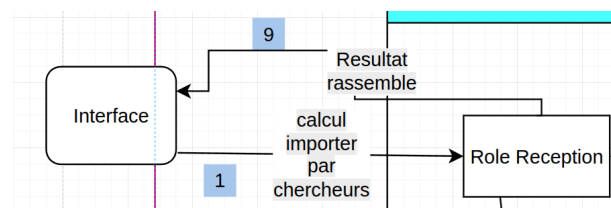


FIGURE 2.14 – Interaction entre le rôle de réception et son environnement

Cette interaction met en évidence la phase d'entrée du système, où les requêtes utilisateur sont capturées et injectées dans le système multi-agents.

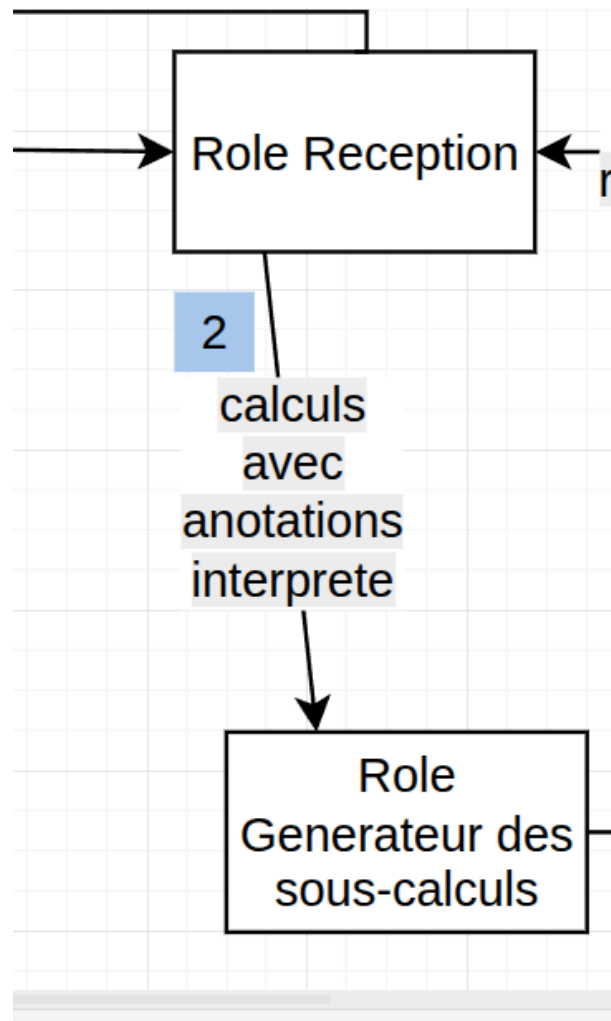


FIGURE 2.15 – Interaction entre le réceptionniste et le générateur de sous-tâches

Cette étape correspond à la transmission de la tâche globale vers le module de décomposition.

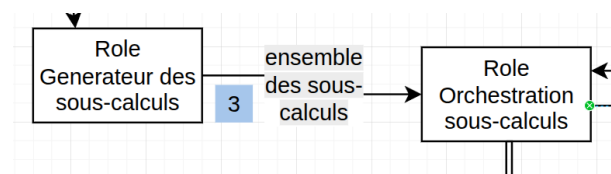


FIGURE 2.16 – Interaction détaillée entre réception et génération des sous-calculs

Elle illustre plus finement le processus de partages des calculs en sous-calculs.

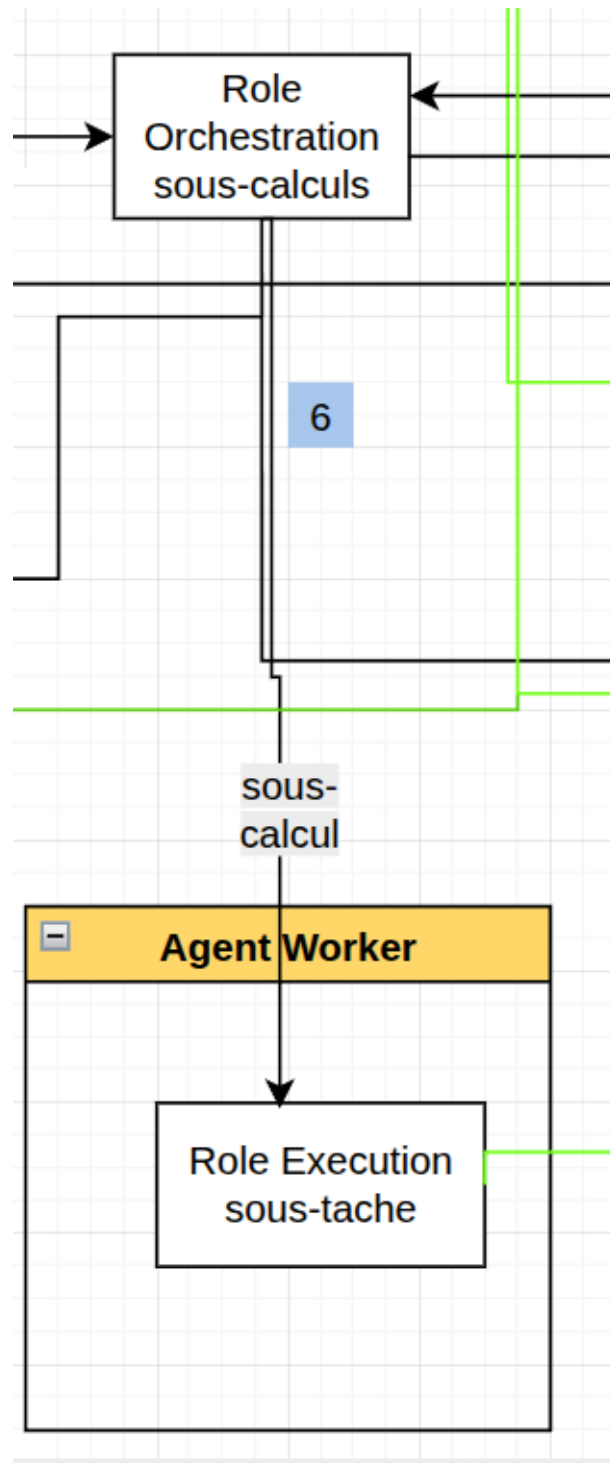


FIGURE 2.17 – Interaction entre le générateur de sous-calculs et l'orchestrateur

Cette interaction met en évidence la phase de planification et d'allocation des ressources.

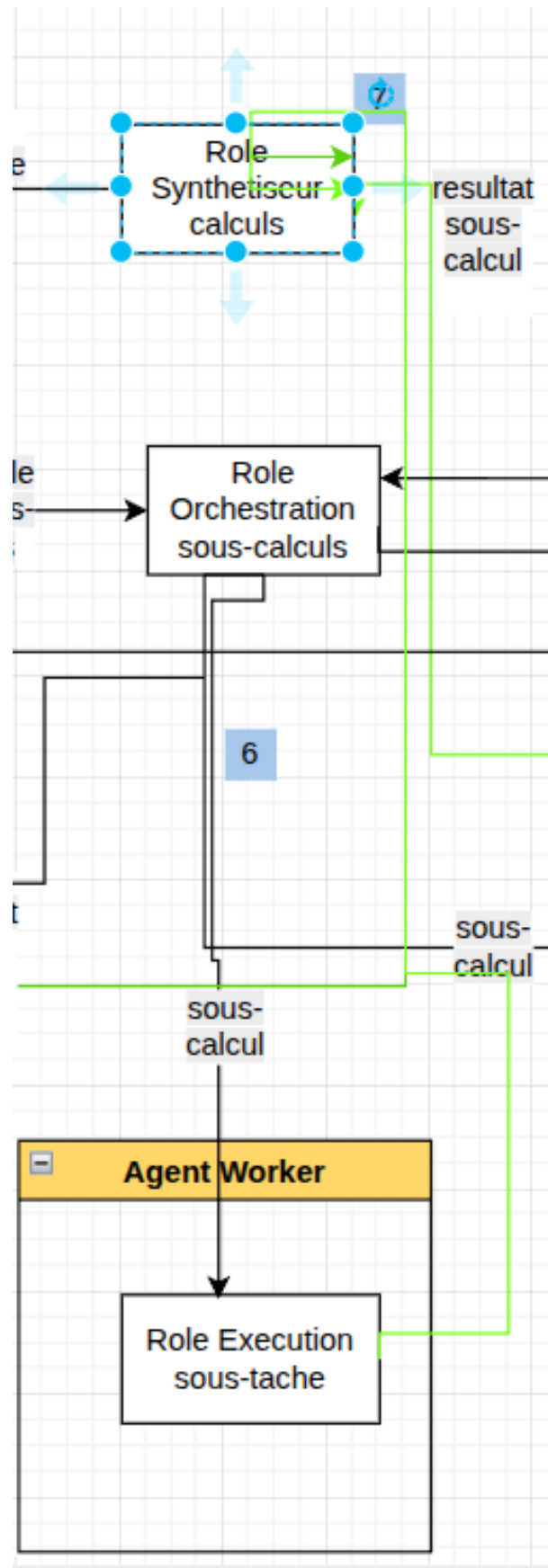


FIGURE 2.18 – Interaction entre les agents d'exécution et le synthétiseur

Elle représente le retour des résultats partiels vers le module de synthèse.

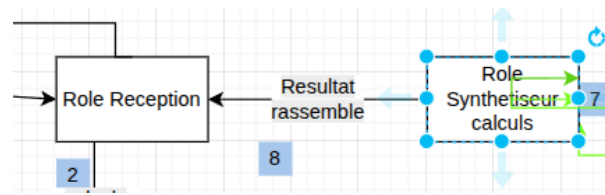


FIGURE 2.19 – Interaction entre le synthétiseur et le réceptionniste

Cette dernière interaction boucle le cycle en transmettant le résultat final vers l'interface utilisateur.

Table des événements du système

Dans le système SMA_PARALLAX, le fonctionnement repose sur un ensemble d'événements discrets déclenchant les actions des agents. Chaque événement correspond à une transition dans le cycle de traitement distribué et peut être associé à un ou plusieurs rôles agentiques.

Le tableau suivant présente les principaux événements identifiés dans le système ainsi que leur description :

N°	Nom de l'événement	Description
E1	Soumission de calcul	Un utilisateur soumet une requête de calcul via l'interface du système. Cet événement déclenche l'activation de l'agent de réception.
E2	Réception de tâche globale	L'agent Master reçoit et valide la tâche globale à traiter.
E3	Décomposition déclenchée	La tâche globale est transformée en sous-calculs exploitables par les agents Worker.
E4	Disponibilité des ressources	Les informations sur l'état des Workers (charge, disponibilité) sont utilisées pour planifier la distribution.
E5	Allocation des sous-calculs	Les sous-tâches sont assignées aux agents Worker selon une stratégie d'orchestration.
E6	Réception d'un sous-calcul	Un agent Worker reçoit une tâche à exécuter.
E7	Début d'exécution	Le Worker commence le traitement du sous-calcul assigné.
E8	Fin d'exécution	Le Worker termine l'exécution du sous-calcul et produit un résultat partiel.
E9	Envoi du résultat partiel	Le résultat calculé est transmis au Master.
E10	Réception des résultats partiels	Le Master collecte les résultats provenant de plusieurs Workers.
E11	Agrégation des résultats	Les résultats partiels sont combinés pour produire un résultat global cohérent.
E12	Production du résultat final	Le résultat final est généré et prêt à être renvoyé à l'utilisateur.
E13	Diffusion d'état (Worker)	Les Workers envoient périodiquement leurs informations d'état (charge, disponibilité).
E14	Collecte des états	L'agent Contrôleur centralise les informations provenant des différents agents.
E15	Détection de défaillance	Une anomalie ou panne d'un nœud est détectée dans le système.
E16	Notification de défaillance	Une alerte est envoyée au Master pour adapter l'orchestration.
E17	Enregistrement d'un nouveau nœud	Un nouvel agent rejoint le système et est intégré dans l'environnement distribué.

TABLE 2.11 – Table des événements du système SMA_PARALLAX

2.2.3 Architecture Agentique

Cette section présente l'architecture agentique du système PARALLAX. Elle décrit l'organisation des différents agents, leurs interactions ainsi que leur rôle dans l'exécution distribuée des tâches. Cette architecture permet de structurer le système en entités autonomes capables de coopérer pour assurer la scalabilité, la robustesse et l'efficacité du calcul distribué.

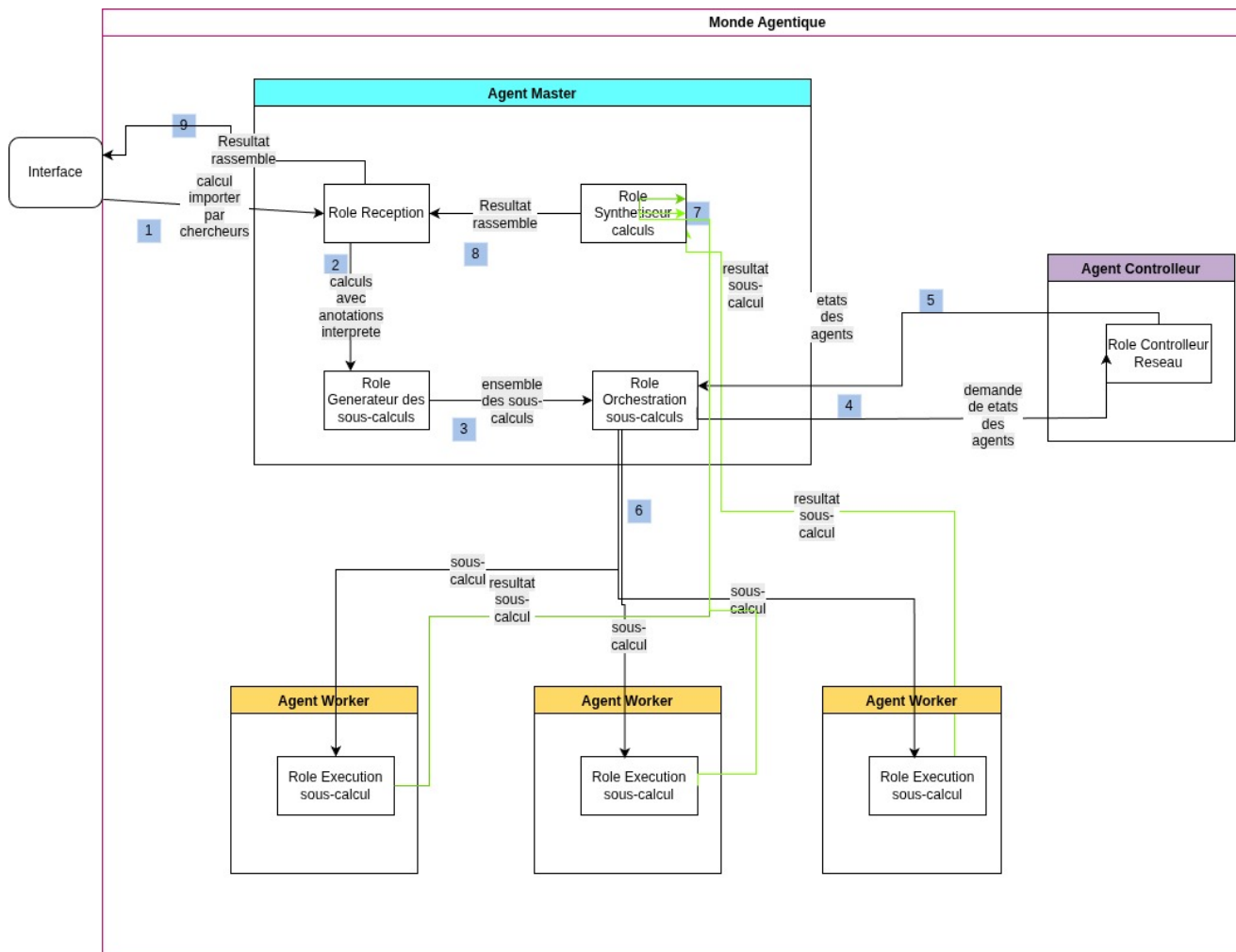


FIGURE 2.20 – Architecture Agentique

2.3 Présentation des Algorithmes Cles

2.3.1 Présentation de l'algorithme d'inscription d'un nouveau nœud

Principe L'inscription d'un nouveau nœud dans le cluster repose sur un processus d'auto-intégration dans lequel le nœud obtient une adresse réseau, génère une identité

unique et se déclare auprès du contrôleur. Une fois validé, il est intégré au système et devient opérationnel pour participer aux calculs distribués.

Entrée

- Nœud N rejoignant le réseau
- Accès au service DHCP
- Accès au canal de diffusion réseau (broadcast)
- Présence du contrôleur du cluster

Algorithm 1 Entrée d'un nœud dans le réseau

Nœud N Nœud intégré au cluster

Début

$N.ip \leftarrow DHCP_REQUEST$

$N.uuid \leftarrow GENERATE_UUID$

STORE_LOCAL($N.uuid$)

SEND_BROADCAST(HELLO, $N.uuid$, $N.ip$)

— **if** RECEIVE(HELLO) par le contrôleur **then** REGISTER_NODE($N.uuid$, $N.ip$)

SEND_ACK($N.uuid$)

CONNECT(N , CONTROLLER)

Fin

Algorithme

Sortie

- Nœud enregistré dans le cluster
- Identifiant unique attribué au nœud ($uuid$)
- Adresse réseau valide (IP assignée)
- Connexion établie avec le contrôleur

2.3.2 Présentation de l'Algorithme d'élection du leader

Principe

2.3.3 Objectif et déclencheurs

Le cluster **PARALLAX** repose sur un mécanisme d'élection permettant de garantir la présence permanente d'un nœud **remplaçant**, chargé d'assurer la continuité du système en cas de défaillance d'un rôle critique. Le remplaçant occupe une position

intermédiaire dans la hiérarchie du cluster : le maître est le plus puissant, le remplaçant est plus puissant que le contrôleur, et le contrôleur est le moins puissant des trois rôles principaux. Le protocole d'élection vise donc à garantir qu'un nouveau remplaçant soit toujours disponible.

Objectif : Le protocole d'élection permet d'atteindre, dans un délai borné et avec un coût en messages maîtrisé, un consensus sur l'identité du nouveau nœud remplaçant du cluster, afin d'assurer la tolérance aux pannes et la continuité du système.

Déclencheurs d'élection : Trois événements principaux peuvent initier le processus d'élection :

- **(D1) Démarrage à froid :** aucun remplaçant n'est présent dans le cluster. Le premier nœud rejoignant le système déclenche automatiquement une élection afin de désigner un remplaçant.
- **(D2) Remplacement du rôle critique :** lorsqu'un maître ou un contrôleur tombe en panne, le remplaçant prend immédiatement sa place. Dans ce cas, un processus d'élection est déclenché pour désigner un nouveau remplaçant.
- **(D3) Stepdown du remplaçant :** le remplaçant actuel se retire (maintenance, surcharge persistante ou réorganisation du cluster). Il libère son rôle et déclenche une réélection.

Dans tous les cas, ces événements déclenchent un processus de réélection garantissant qu'un nouveau remplaçant est désigné de manière cohérente, assurant ainsi la continuité de la hiérarchie du cluster et la résilience du système.

Fonction de score Chaque nœud i calcule un score représentant sa capacité globale à être désigné comme remplaçant. Ce score combine plusieurs dimensions : ressources de calcul, mémoire disponible, performance réseau et fréquence CPU.

$$score_n_i = a \cdot \frac{CPU_{libre} \cdot CPU_{total}}{CPU_{libre}^{max} \cdot CPU_{total}^{max}} b \cdot \frac{RAM_{libre} \cdot RAM_{total}}{RAM_{libre}^{max} \cdot RAM_{total}^{max}} c \cdot \frac{LATENCE_{min}}{LATENCE_i} d \cdot \frac{freq_i}{freq_{max}}$$

Signification des paramètres :

- CPU_{libre} : fraction de cœurs CPU actuellement disponibles sur le nœud i .
- CPU_{total} : nombre total de cœurs CPU du nœud i .
- CPU_{libre}^{max} : valeur(fraction) maximale de CPU libre observée dans le cluster, telle qu'émise et maintenue par le contrôleur (heartbeat global).
- CPU_{total}^{max} : capacité maximale en CPU total parmi tous les nœuds du cluster.
- RAM_{libre} : fraction de mémoire RAM disponible sur le nœud i .
- RAM_{total} : quantité totale de RAM du nœud i .

- RAM_{libre}^{max} : maximum de fraction de RAM libre observé dans le cluster, calculé et diffusé par le contrôleur.
- RAM_{total}^{max} : capacité maximale de RAM totale parmi tous les nœuds.
- $LATENCE_i$: latence réseau mesurée du nœud par le contrôleur i .
- $LATENCE_{min}$: latence minimale observée dans le cluster (référence optimale).
- $freq_i$: fréquence CPU du nœud i .
- $freq_{max}$: fréquence CPU maximale observée dans le cluster.

Entrées

- Ensemble des nœuds disponibles V_t
- Pour chaque nœud i :
 - CPU_{libre}, CPU_{total}
 - RAM_{libre}, RAM_{total}
 - $LATENCE_i$
 - $freq_i$
- Valeurs globales maintenues par le contrôleur :
 - $CPU_{libre}^{max}, CPU_{total}^{max}$
 - $RAM_{libre}^{max}, RAM_{total}^{max}$
 - $LATENCE_{min}$
 - $freq_{max}$
- Paramètres de pondération :

$$a = 0,35, \quad b = 0,35, \quad c = 0,15, \quad d = 0,15$$

Algorithme

Sorties

- Identifiant du leader élu $Leader_t^*$
- Mise à jour des états des nœuds :
 - *leader* pour le nœud élu
 - *normal* pour les autres
- Diffusion du message *coord*

Algorithm 2 Initiation d'une élection par le nœud k

```
1:  $\sigma_k \leftarrow electing$ 
2:  $H \leftarrow \{j \in V \mid Score_{j,t} > Score_{k,t}\}$ 
3: if  $H = \emptyset$  then
4:   diffuser  $coord_{uuid_k}$ 
5:    $\sigma_k \leftarrow leader$ 
6: else
7:   for all  $j \in H$  do
8:     envoyer  $electionScore_{k,t}, uuid_k$  à  $j$ 
9:   end for
10:  armer temporisation  $T_{attente}$ 
11: end if
```

Algorithm 3 Traitement des messages d'élection

```
1: procedure ONRECEIVEELECTION( $s, u$ )
2:   if  $Score_{k,t} > s$  ou ( $Score_{k,t} = s$  et  $uuid_k > u$ ) then
3:     envoyer  $alive$  à l'émetteur
4:     INITIALEELECTION( $k$ )
5:   end if
6: end procedure
7: procedure ONRECEIVECOORD( $u$ )
8:    $leader_k \leftarrow u$ 
9:    $\sigma_k \leftarrow normal$ 
10: end procedure
```

2.3.4 Présentation de l'Algorithme de Décomposition des tâches et recomposition

Introduction Dans un système de calcul distribué, la **décomposition des calculs** et la **recomposition** constituent les mécanismes fondamentaux permettant d'exploiter efficacement les ressources de plusieurs machines.

Nous commençons par définir la notion de **calcul élémentaire**.

Définition 2.3.1. Une **calcul élémentaire** est un calcul qui ne peut plus être décomposé en sous-calculs de manière profitable. Autrement dit, la décomposition d'un tel calcul coûterait plus cher (en temps de communication, de synchronisation et de gestion) que son exécution sur une seule machine.

L'objectif du système de calcul distribué est de pouvoir découper un calcul complexe en sous-calculs qui peuvent s'exécuter de manière **pseudo-indépendante** sur des machines distinctes, puis d'agréger les résultats de manière à ce que le workflow global ne subisse pas de dégradation de performance significative par rapport à une exécution séquentielle.

2.3.5 Types de décomposition des calculs

Il existe deux approches principales pour la décomposition d'un calcul :

Décomposition par les données (Data Splitting)

Soit un ensemble d'instructions I qui s'exécute sur un ensemble de données D . La décomposition par les données consiste à découper D en n fragments $D_1, D_2, \dots, D_n$, à exécuter les mêmes instructions I sur chaque fragment de manière indépendante, puis à agréger les résultats partiels pour obtenir le résultat final.

Cette approche est particulièrement adaptée aux programmes qui traitent de **très grands volumes de données** ou qui présentent une forte indépendance des données, par exemple :

- L'entraînement de modèles sur des sous-ensembles de données (data parallelism)
- Le balayage d'hyperparamètres (hyperparameter tuning)
- Les simulations Monte-Carlo
- Le traitement d'images ou de vidéos par lots

Décomposition par les instructions (Instruction Splitting)

Dans ce cas, on dispose d'un ensemble d'instructions I composé de sections **pseudo-indépendantes** (ou totalement indépendantes). On découpe I en sous-ensembles $I_1, I_2, \dots, I_n$, que l'on exécute sur des machines différentes, puis on agrège les résultats.

Cette approche convient particulièrement aux programmes contenant des sections de code pouvant être exécutées en parallèle, telles que :

- Les boucles `for` indépendantes (embarrassingly parallel)
- Les calculs sur des branches indépendantes d'un arbre de décision
- Les calculs de gradients indépendants dans certains algorithmes d'optimisation
- Les simulations de scénarios indépendants

2.3.6 Hypothèse de conception

Puisque la manière optimale de décomposer un programme dépend fortement de sa structure interne, nous faisons l'hypothèse suivante :

L'utilisateur (chercheur/développeur) connaît parfaitement son code et est capable d'identifier les opportunités de parallélisation (par données ou par instructions) pour chaque partie du programme.

Sous cette hypothèse, il devient nécessaire de fournir un mécanisme permettant à l'utilisateur de **communiquer ses intentions de calcul distribué** au système. C'est pourquoi nous introduisons un système d'annotations.

2.3.7 Système d'annotations

Une **annotation** est un préfixe spécial placé dans le code source qui exprime l'intention du programmeur concernant l'exécution distribuée du calcul.

Annotation	Utilisation	Paramètres
<code>@parallax.split(params)</code>	Indique qu'un calcul doit être décomposé et distribué	type : data instructions type=null → région parallèle exécutée sur un seul nœud ncpu : nombre de CPUs requis ram : mémoire minimale requise aggregate : fonction de re-composition des résultats
<code>@parallax.dag</code>	Définit une région contenant un graphe de dépendances acyclique (DAG) entre sous-calculs parallèles	-
<code>@parallax.shared</code>	Spécifie les variables devant être partagées entre les nœuds de calcul	Liste de variables

TABLE 2.12 – Table des annotations du système Parallax

2.3.8 Algorithme de Parsing

Le **parser d'annotations** est le composant chargé d'interpréter les annotations présentes dans le code source et de les transformer en structures exécutables permettant la décomposition et la recombinaison des calculs.

Fonctionnement général du parser

Principe Le parser a pour rôle de transformer un programme annoté en une représentation exécutable distribuée. Il analyse la structure syntaxique du code, identifie les annotations liées au calcul distribué, puis génère les mécanismes nécessaires à la décomposition, à la distribution et à la recomposition des calculs.

Entrée

- Programme source annoté
- Arbre Syntaxique Abstrait (AST) du programme
- Ensemble des règles d'annotations (`@parallax.split`, `@parallax.dag`, etc.)

Algorithm 4 Fonctionnement général du parser

- 1: Analyser l'**Arbre Syntaxique Abstrait (AST)** du programme
 - 2: Détecter les annotations présentes dans le code
 - 3: Attribuer un **identifiant unique** (`annotation_id`) à chaque annotation
 - 4: Injecter des fonctions auxiliaires pour la décomposition, la distribution et la recomposition des calculs
-

Algorithme

Sortie

- Programme transformé et instrumenté
- Structure annotée enrichie avec des `annotation_id`
- Fonctions auxiliaires injectées pour l'exécution distribuée

2.3.9 Interprétation des annotations

Annotation `@parallax.split` Lorsqu'une annotation `@parallax.split` est associée à une fonction f , le parser génère deux mécanismes principaux :

* **Phase de décomposition et distribution :**

- Interrogation du registre de services pour identifier les nœuds disponibles.
- Calcul de la répartition des charges selon :

$$w_i = \frac{x_i}{\sum_{j=1}^n x_j}$$

où x_i représente la capacité du nœud i (CPU, RAM, etc.).

* **Phase de synchronisation et recomposition :**

- Attente de la fin d'exécution de tous les sous-calculs (via barrière ou futex).
- Application de la fonction aggregate définie par l'utilisateur afin de produire le résultat global.

Annotation @parallax.dag Cette annotation définit une région contenant un **graphe orienté acyclique (DAG)** représentant les dépendances entre sous-calculs. Le parser construit dynamiquement ce graphe à partir des dépendances explicites et l'utilise pour déterminer un ordonnancement valide des exécutions via un tri topologique.

Annotation @parallax.shared Cette annotation permet de déclarer explicitement les variables devant être partagées entre les différents nœuds de calcul afin d'assurer la cohérence des données lors de l'exécution distribuée.

2.3.10 Processus de décomposition et de recomposition des calculs

Cette section décrit le processus complet d'exécution distribuée, depuis la réception d'un programme par le nœud maître jusqu'à la production du résultat final. Ce processus est structuré en deux phases principales : la **décomposition des calculs** et la **recomposition des résultats**.

Algorithme de décomposition des calculs

Principe L'algorithme repose sur un modèle maître-worker dans lequel un programme est analysé, partitionné en sections distribuées et non distribuées, puis exécuté de manière hybride. Les sections distribuées sont envoyées aux workers après une phase de planification basée sur l'état des ressources du système.

Entrée

- Programme P reçu par le nœud maître
- Ensemble des annotations du programme
- État des ressources des nœuds (via Agent Controlleur)

Algorithm 5 Processus de décomposition des calculs

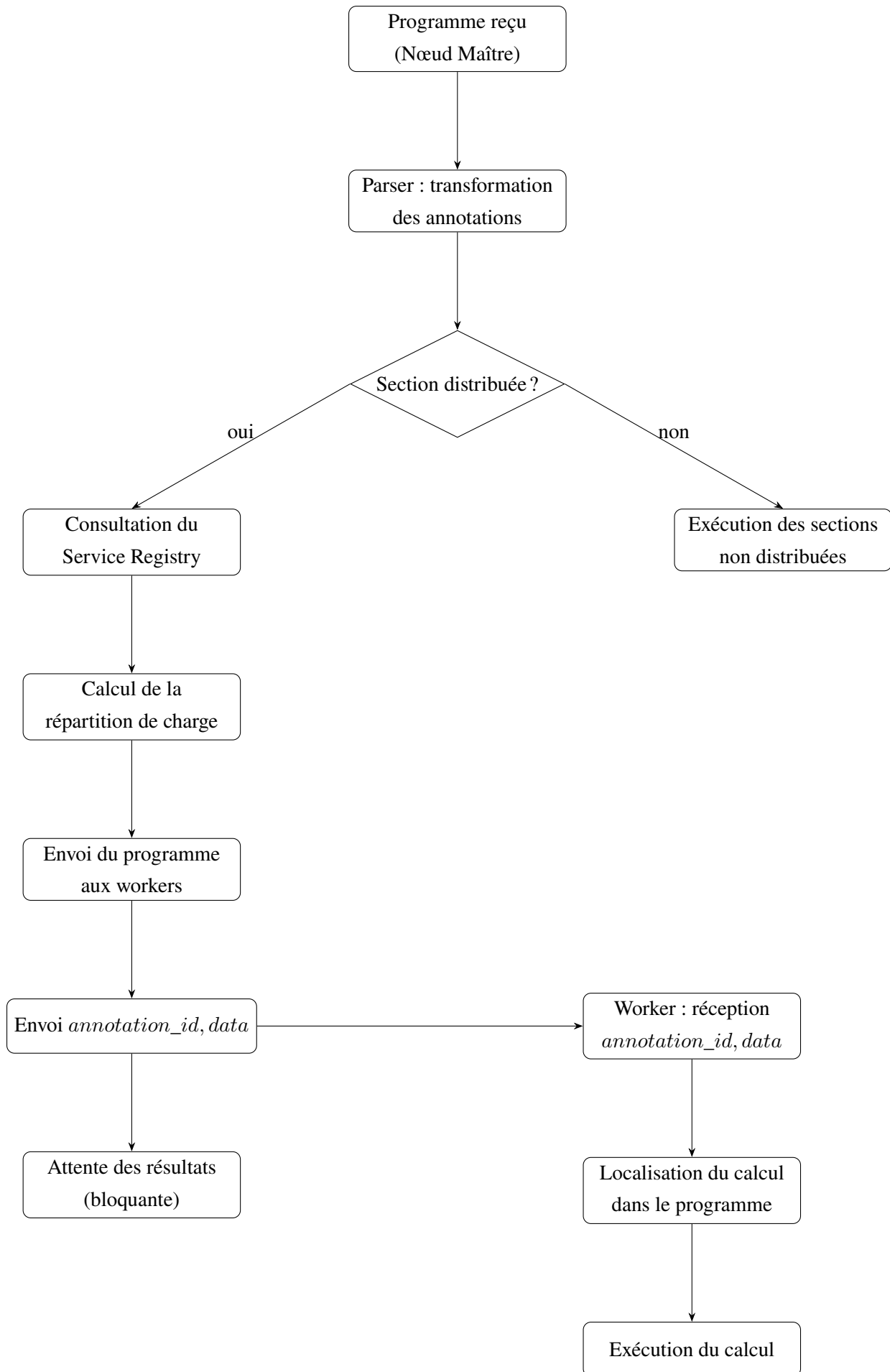
- 1: Parser le programme et remplacer les annotations par les fonctions auxiliaires correspondantes
 - 2: Exécuter séquentiellement les sections non distribuées
 - 3: **for** chaque section S marquée comme distribuée **do**
 - 4: Interroger l'agent contrôleur pour obtenir l'état des nœuds disponibles
 - 5: Calculer la répartition de charge selon la formule de pondération
 - 6: Envoyer une copie du programme instrumenté aux workers sélectionnés
 - 7: Envoyer à chaque worker le couple $annotation_id, data$
 - 8: **end for**
 - 9: Attendre la réception de tous les résultats via un mécanisme de synchronisation (futex ou équivalent)
-

Algorithme**Sortie**

- Ensemble des résultats partiels provenant des workers
- Résultat global prêt pour la phase de recomposition
- État synchronisé du nœud maître après complétion de l'exécution distribuée

Comportement des workers Chaque worker exécute le processus suivant :

- Réception du couple $annotation_id, data$
- Localisation de la section de calcul correspondante dans le programme reçu
- Exécution du calcul sur les données associées
- Envoi du résultat partiel au nœud maître



Algorithme de recomposition des calculs

Processus de recomposition des calculs

Principe La recomposition intervient après la terminaison de l'exécution de tous les sous-calculs par les workers. Elle consiste à synchroniser les résultats partiels et à les agréger afin de produire le résultat global du calcul.

Entrée

- Résultats partiels provenant des workers
- Compteur de complétion (`thread_task_completed_num`)
- Fonction d'agrégation `aggregate`

Algorithm 6 Processus de recomposition des calculs

- 1: Réceptionner les résultats partiels des workers
 - 2: Mettre à jour le compteur de complétion (`thread_task_completed_num`) via mécanisme de synchronisation (`futex`)
 - 3: **if** tous les résultats attendus sont reçus **then**
 - 4: Appliquer la fonction `aggregate` définie par l'utilisateur
 - 5: Combiner les résultats partiels pour produire le résultat global
 - 6: **else**
 - 7: Attendre la réception des autres résultats
 - 8: **end if**
-

Algorithme

Sortie

- Résultat global du calcul
- État synchronisé du nœud maître
- Confirmation de complétion de l'exécution distribuée

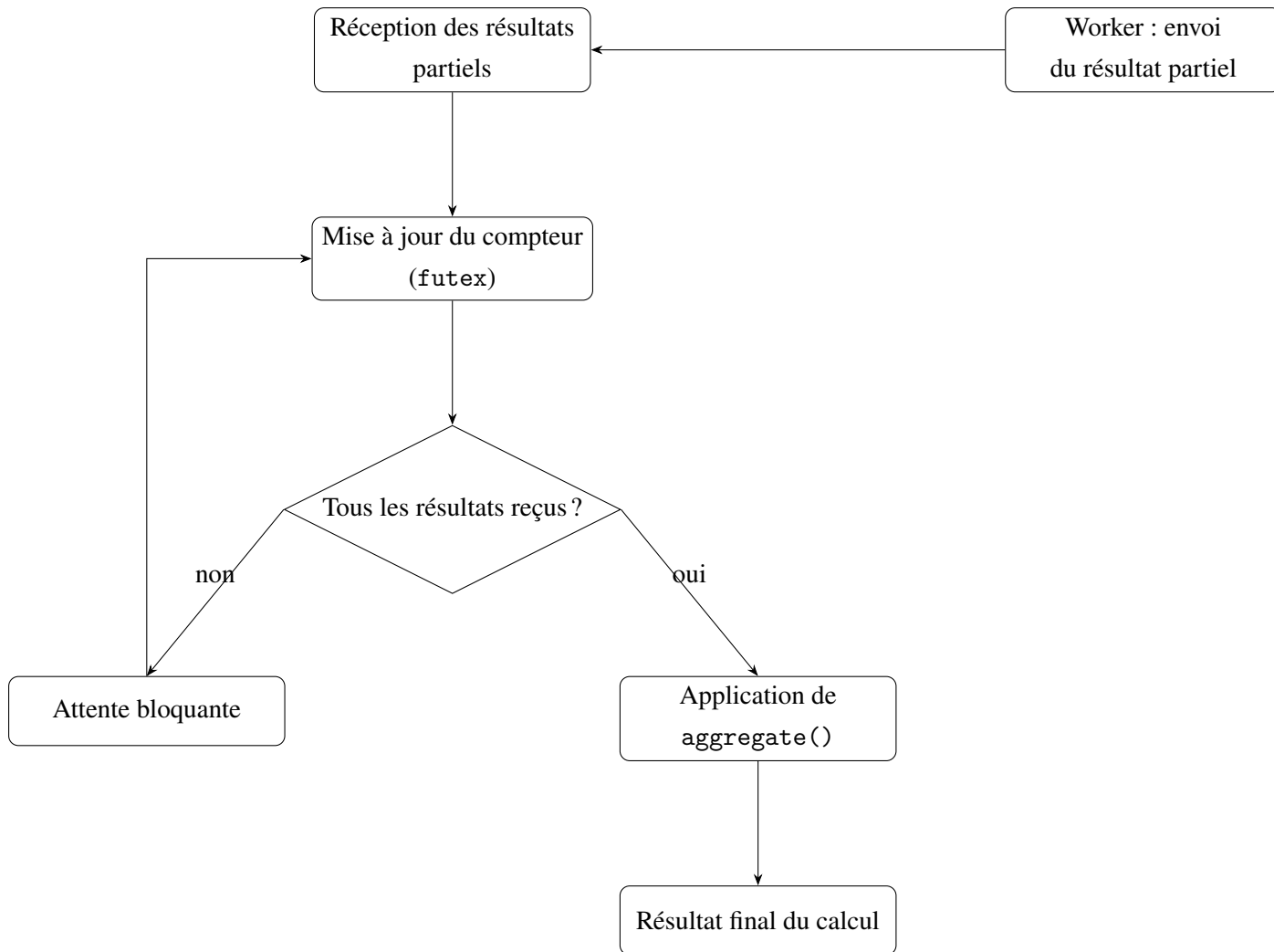


FIGURE 2.22 – Workflow de recomposition des calculs

2.3.11 Présentation de l’algorithme de signalisation par heartbeat

Rôles du heartbeat

- Le heartbeat signale la *présence* d’un nœud.
- Il transporte la *télémétrie* nécessaire à l’orchestration (charges CPU et RAM, longueur de la file de tâches, score courant).
- Son absence déclenche la *détection de panne*.
- Les métriques véhiculées permettent d’identifier les cas de *surcharge*.

Architecture à deux couches

Nous retenons une architecture hybride combinant centralisation et décentralisation.

Couche 1 — heartbeat unicast vers le maître. Chaque nœud n_i envoie, toutes les T_{HB} secondes, un heartbeat unicast vers le maître sur le canal best-effort C_{be} .

Couche 2 — gossip inter-pairs. Toutes les T_G secondes (avec $T_G > T_{HB}$), chaque nœud envoie à k pairs choisis uniformément au hasard un *résumé gossip* contenant sa vue récente du cluster.

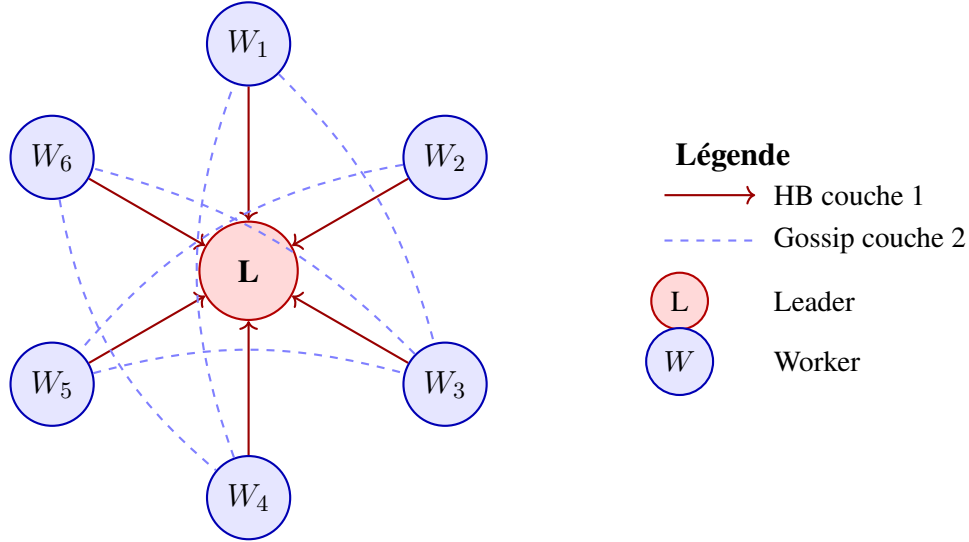


FIGURE 2.23 – Architecture à deux couches du heartbeat.

Structure d'un message de heartbeat

$$HB_i, t = (\text{uuid}_i, k_i t, \widehat{CPU}_i t, \widehat{RAM}_i t, q_i t, \text{Score}_i, t, \sigma_i, L_i t)$$

Paramètres temporels

TABLE 2.13 – Paramètres du protocole de heartbeat

Paramètre	Valeur	Signification
T_{HB}	2 s	Période du heartbeat unicast
T_G	5 s	Période du gossip inter-pairs
k	$\lceil \log_2 N \rceil$	Fanout du gossip
T_{suspect}	4 s	Seuil de passage <code>ALIVE</code> $\rightarrow$ <code>SUSPECTED</code>
T_{failed}	8 s	Seuil de passage <code>SUSPECTED</code> $\rightarrow$ <code>FAILED</code>
θ_{high}	0,85	Seuil haut de charge
θ_{low}	0,65	Seuil bas de charge (hystérésis)

Détection de panne en deux phases

Phase 1 — Suspicion. Soit $\Delta_{ij}t$ l'écart depuis le dernier heartbeat reçu de j . Si $\Delta_{ij}t \geq T_{\text{suspect}}$, le nœud i marque j comme SUSPECTED et émet un message GOSSIP_QUERY à ses k voisins.

Phase 2 — Défaillance confirmée. Si $\Delta_{ij}t \geq T_{\text{failed}}$ et si la majorité des réponses à la GOSSIP_QUERY sont négatives, alors i marque j comme FAILED et déclenche une élection si j était le leader.

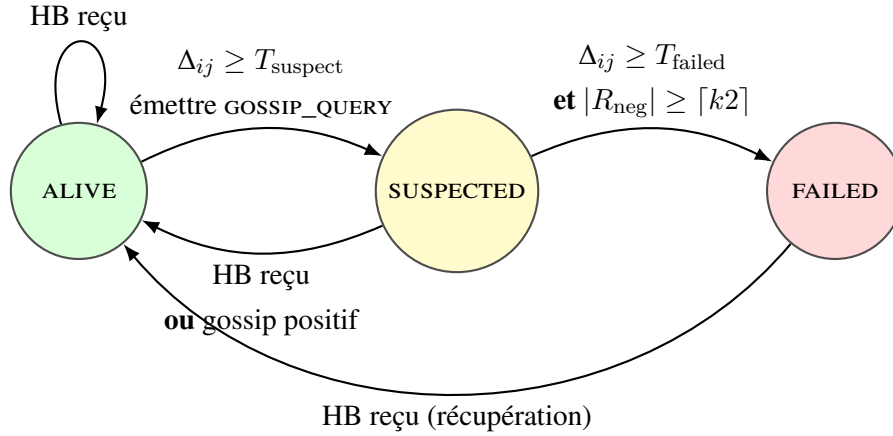


FIGURE 2.24 – Machine à états du suivi d'un pair par un observateur.

2.3.12 Modélisation UML

Diagramme de classes métier

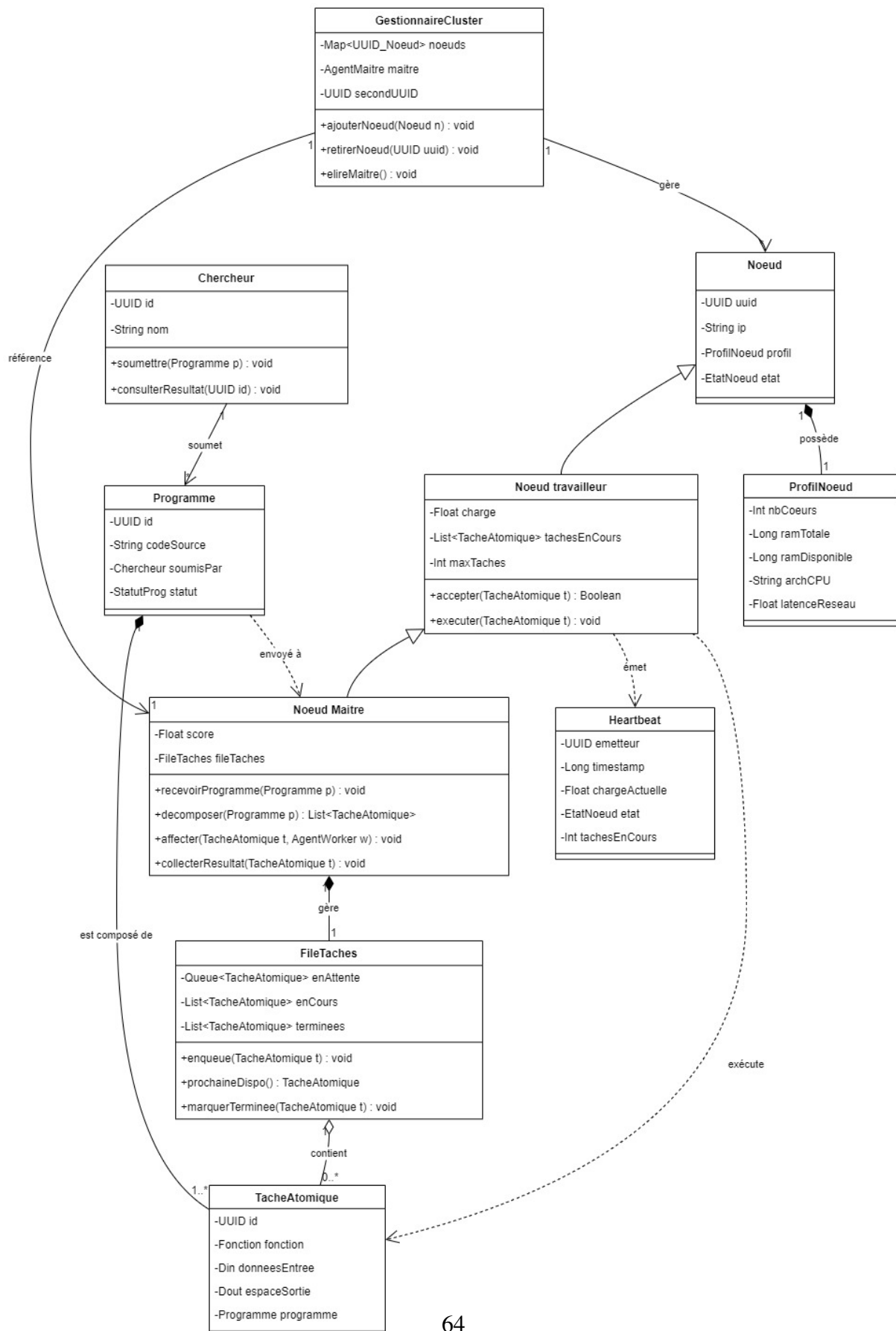


FIGURE 2.25 – Diagramme de classe metiers

diagramme de sequences Techniques

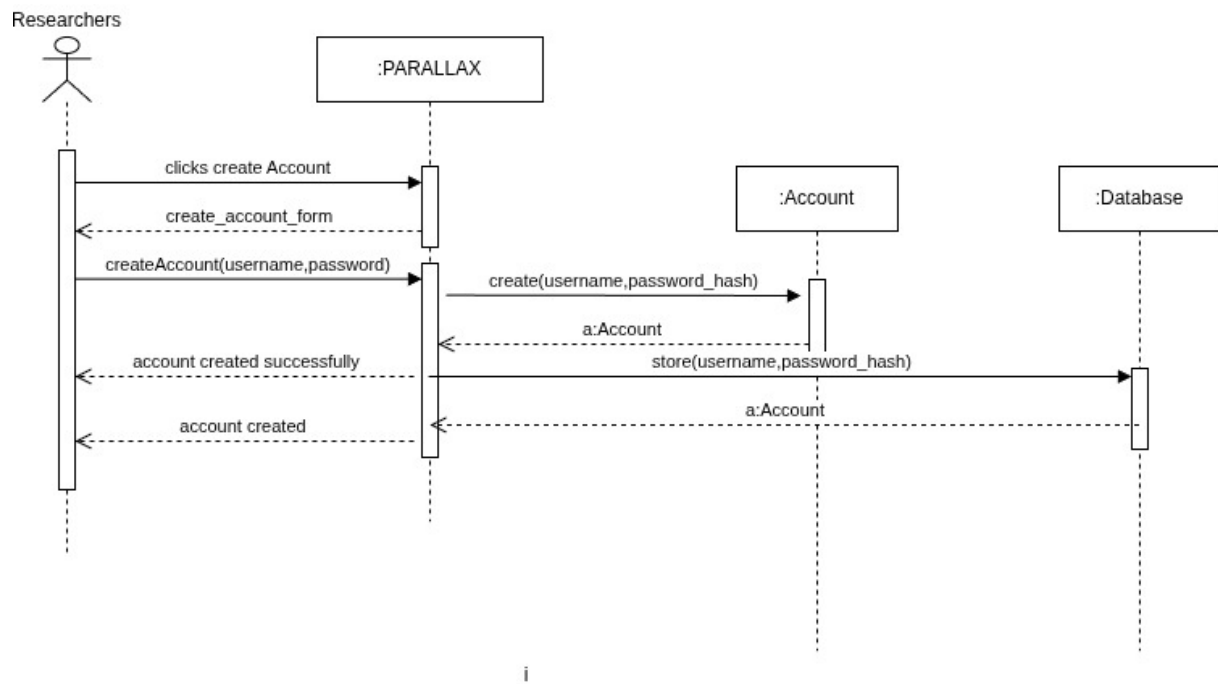


FIGURE 2.26 – parallax-AUTHENTIFICATION

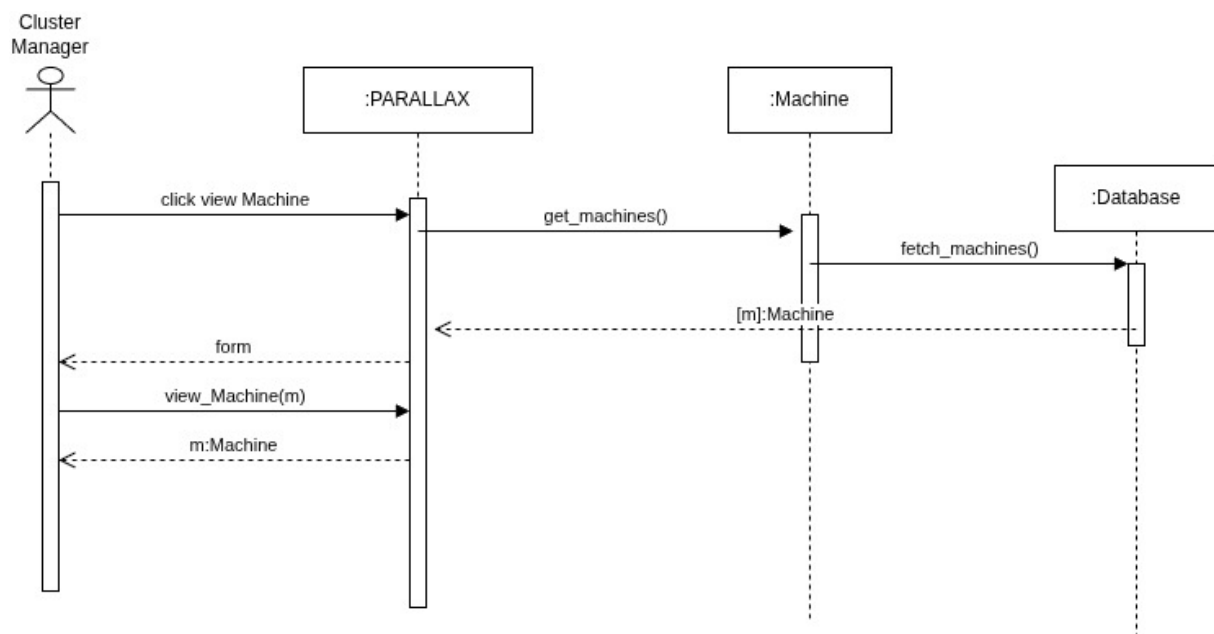


FIGURE 2.27 – consulter machines

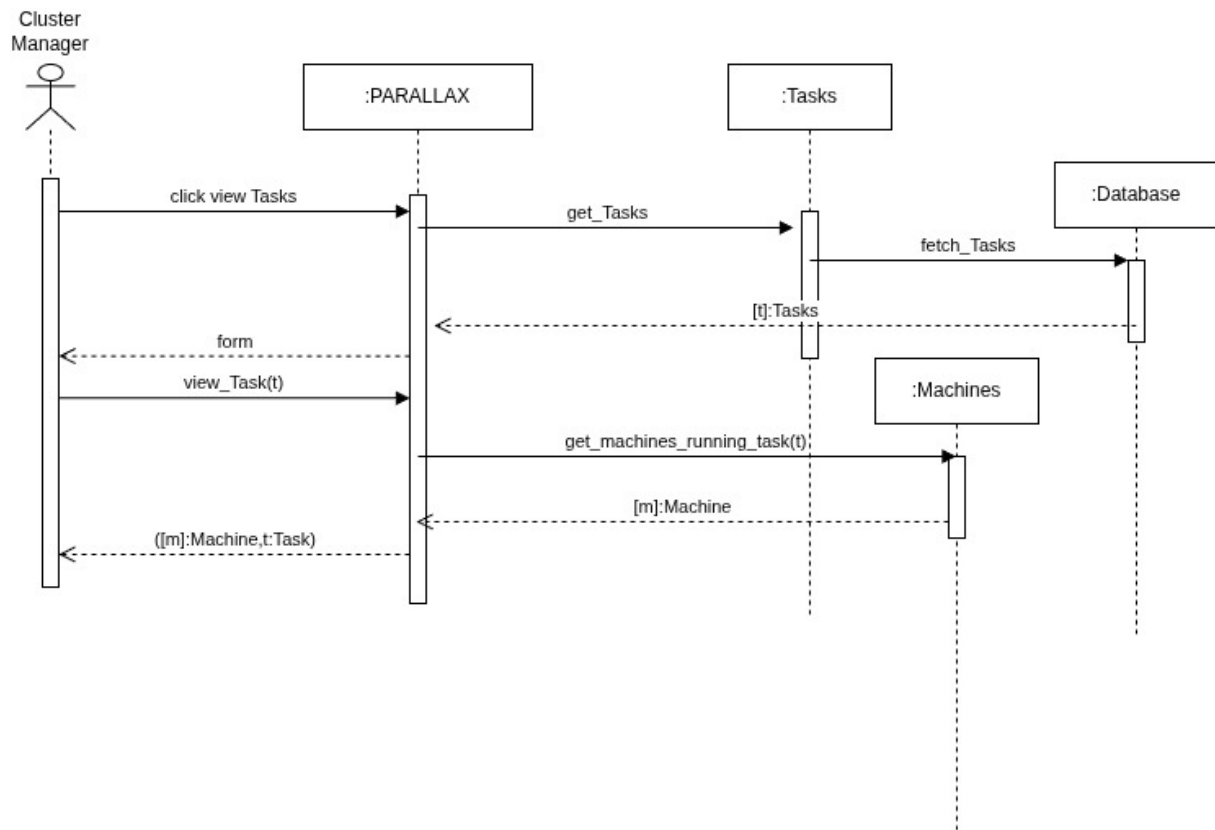


FIGURE 2.28 – consulter taches

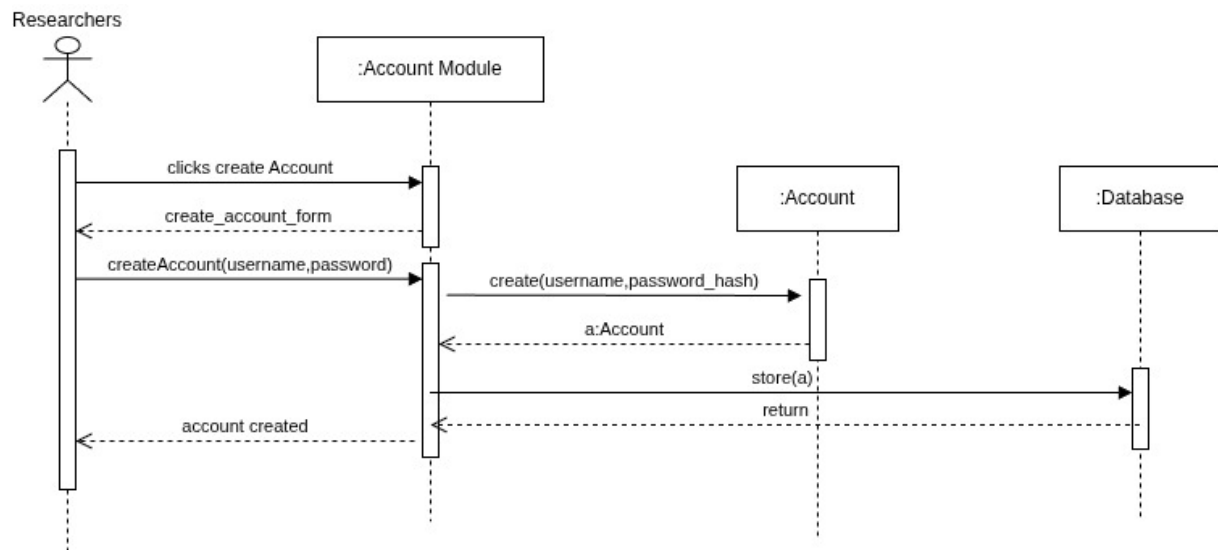


FIGURE 2.29 – CREATE ACCOUNT

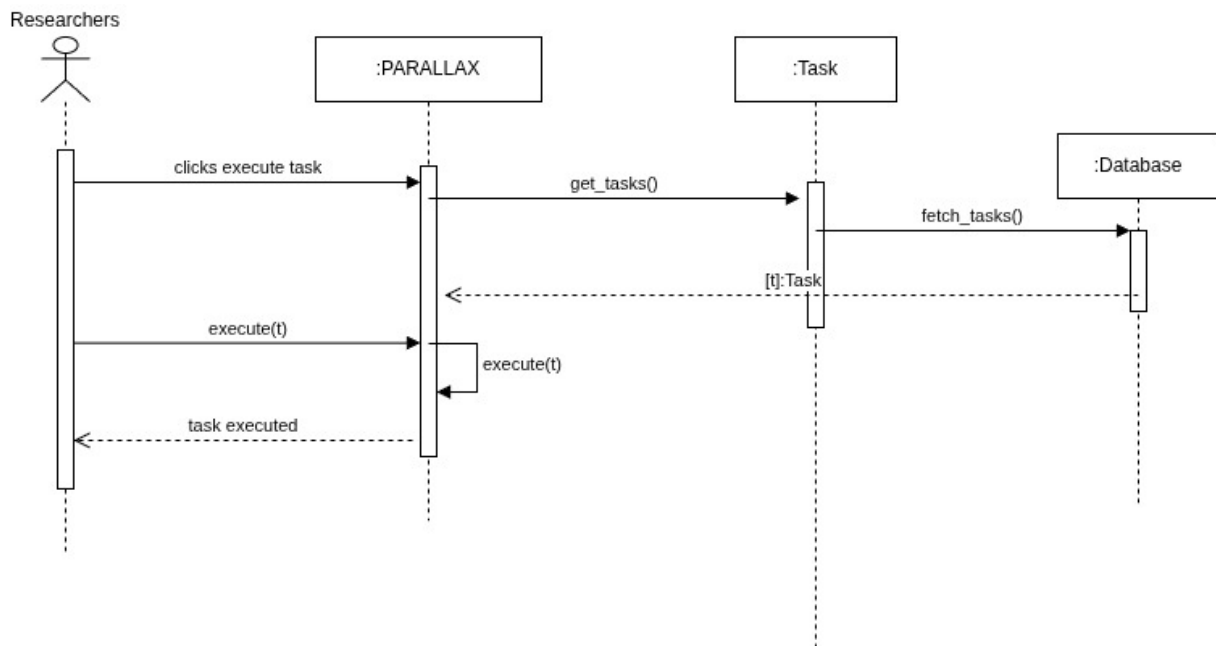


FIGURE 2.30 – Execution de tâche

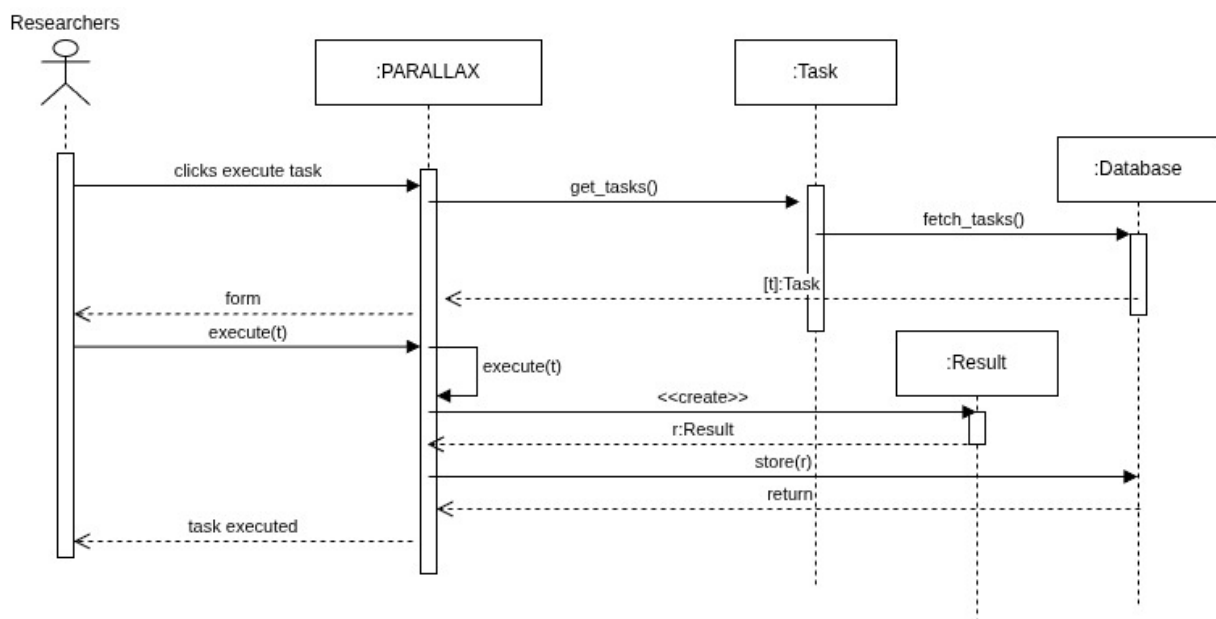


FIGURE 2.31 – importer des tâches

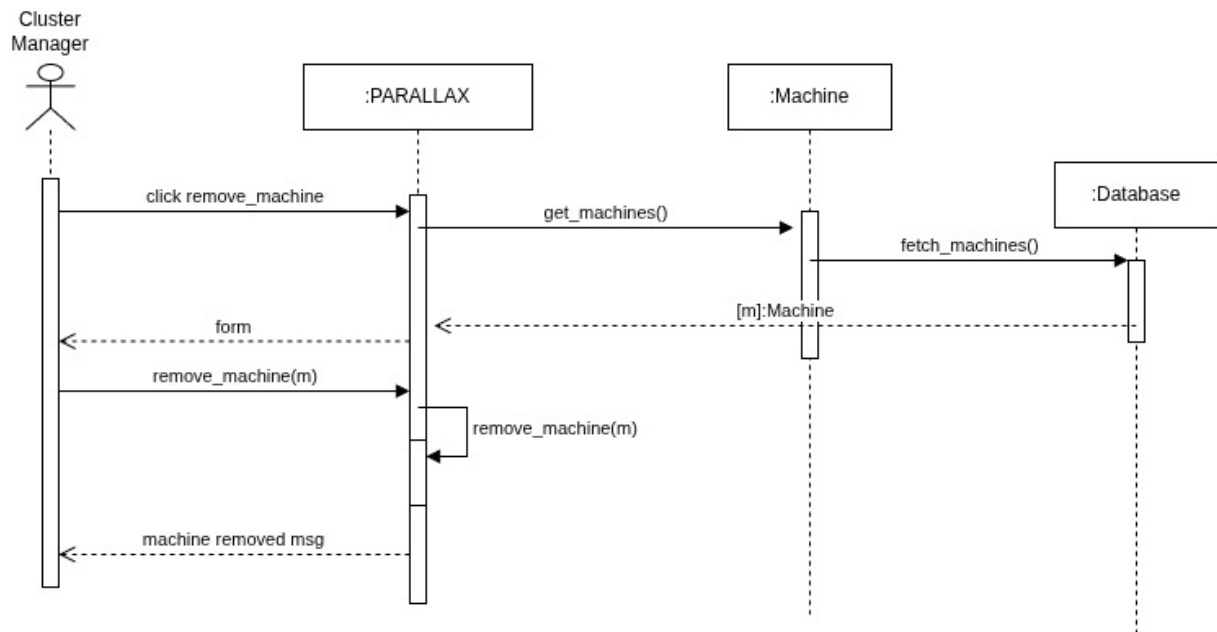


FIGURE 2.32 – enlever un noeud

diagramme d'états d'une tâche

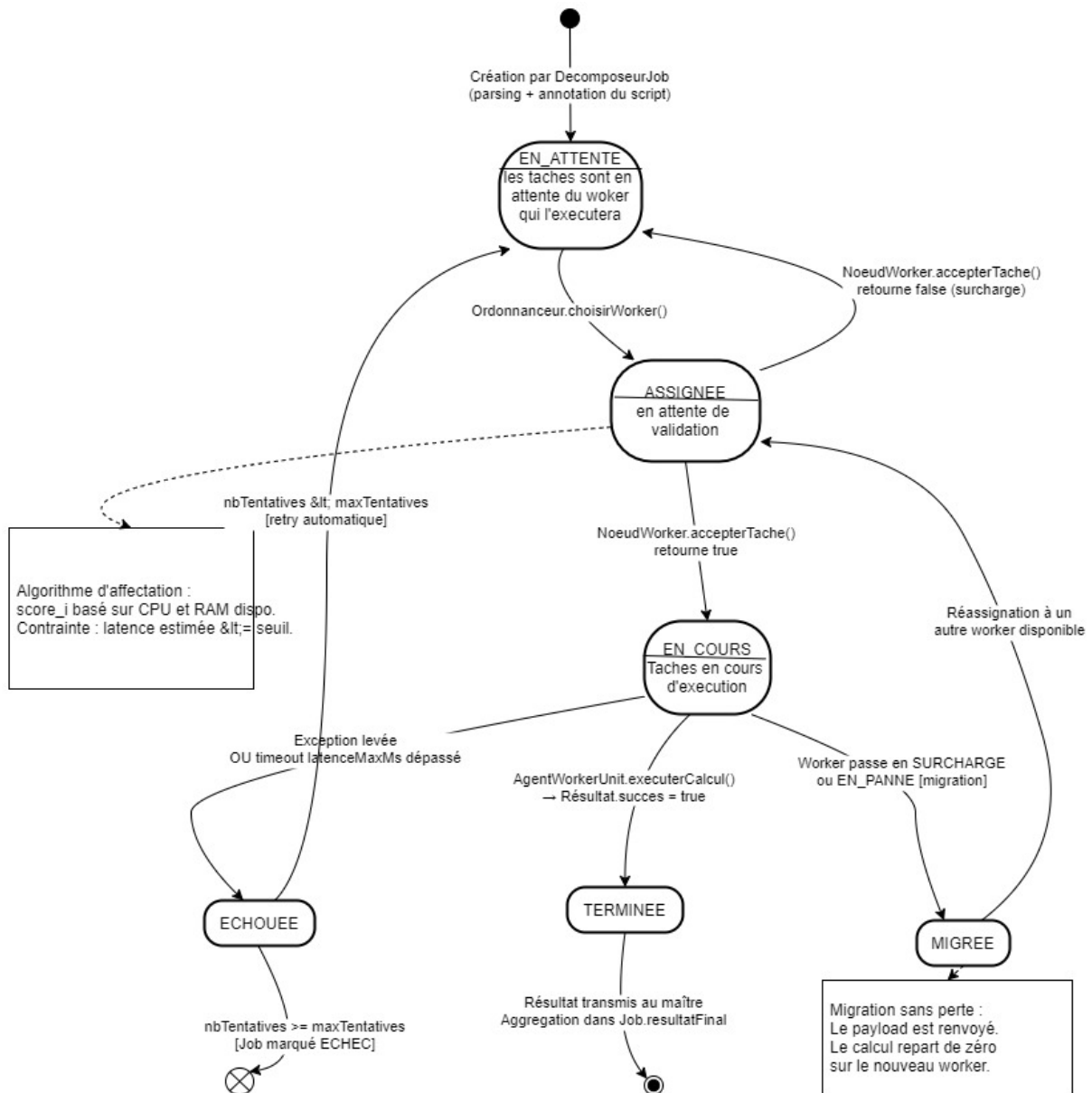


FIGURE 2.33 – Diagramme d'état d'une tâche

diagramme d'états d'un noeud

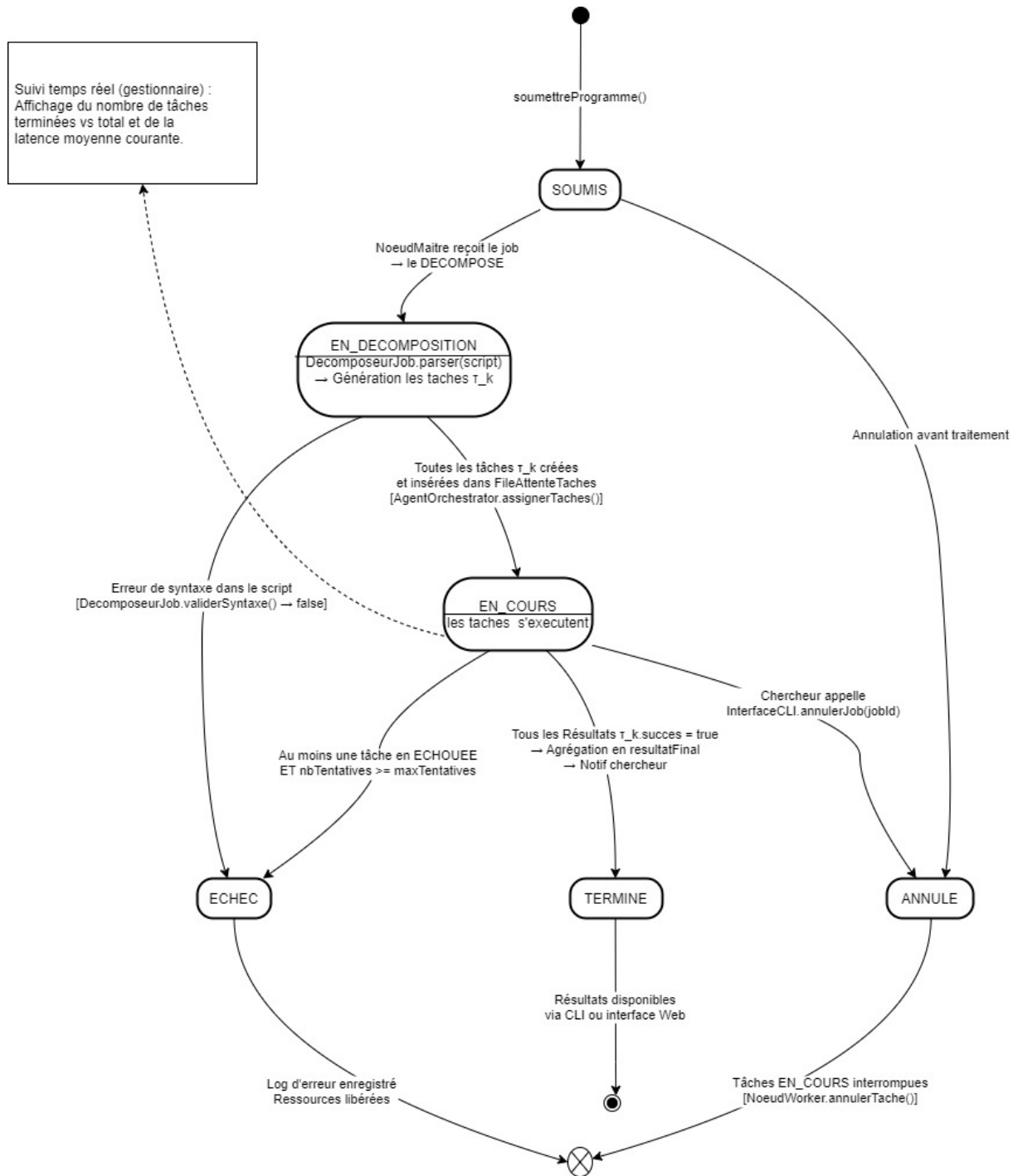


FIGURE 2.34 – Diagramme d'état d'un programme

2.3.13 Diagramme d'états

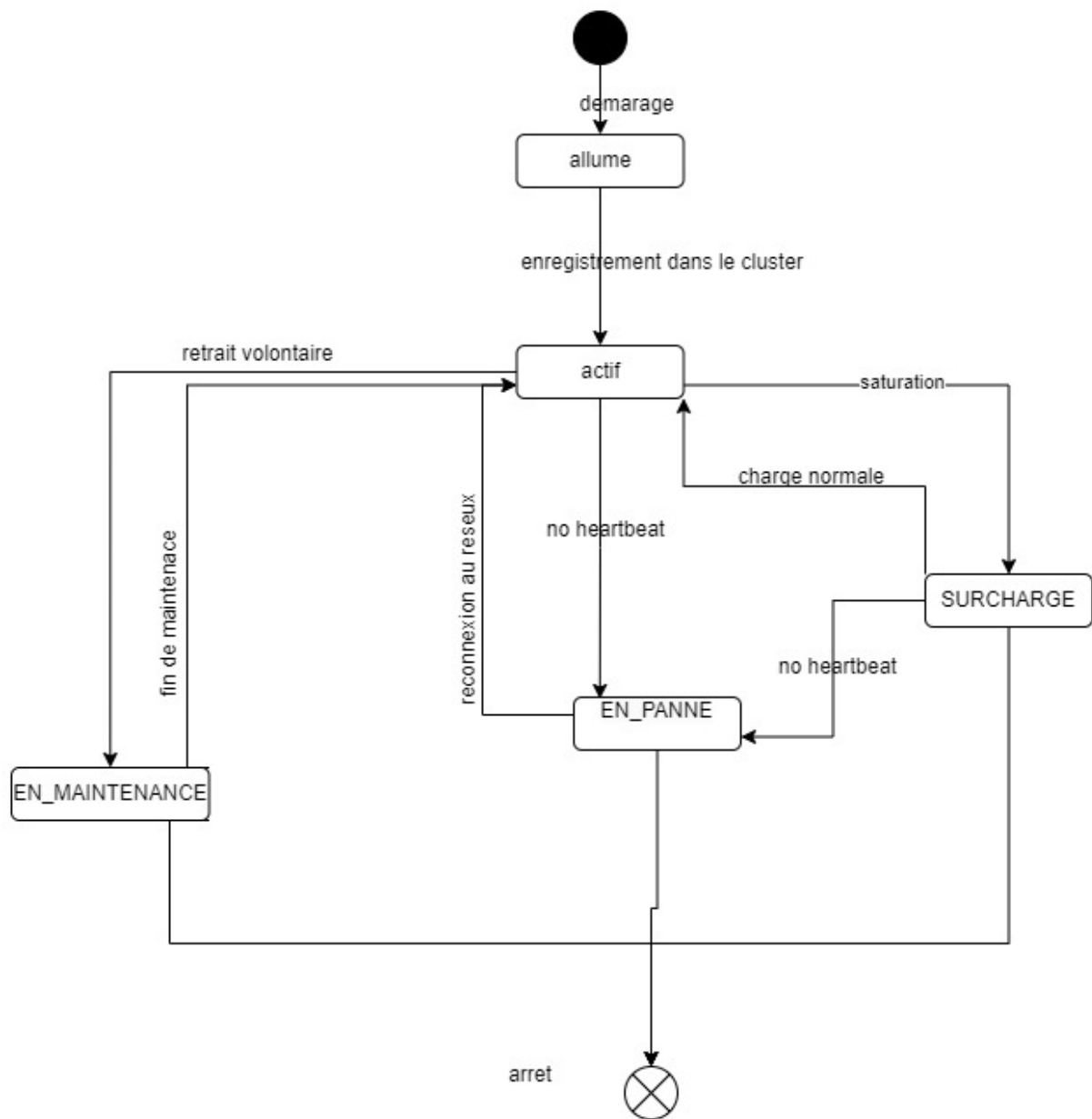


FIGURE 2.35 – Diagramme d'état d'un nœud

2.3.14 Diagramme des composants

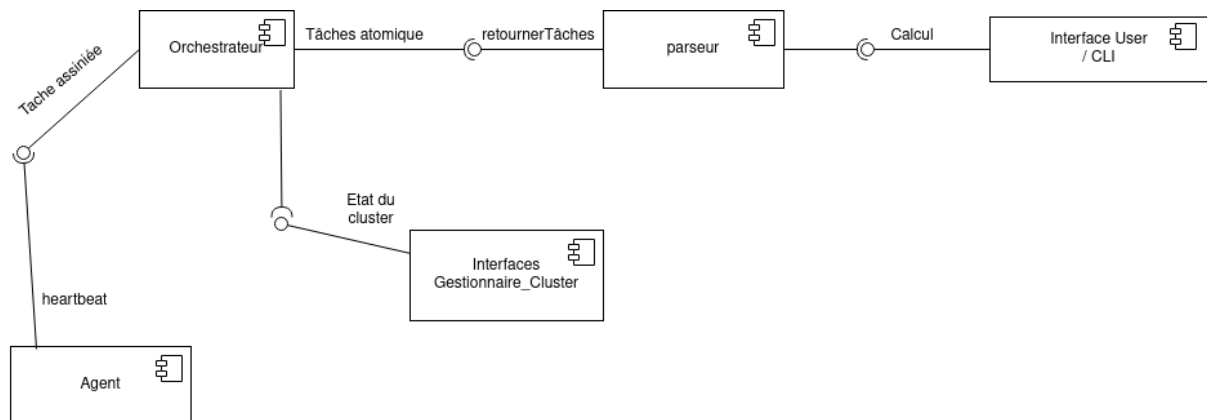


FIGURE 2.36 – Diagramme des composants - parallax

Chapitre 3

Implémentation et Résultats

3.1 Outils et Technologies

3.2 Diagramme de Déploiement

3.3 Résultats

Conclusion Générale

Références

Bibliographie

- [1] D. P. Anderson, “SETI@home : An Experiment in Public-Resource Computing,” *Communications of the ACM*, vol. 45, no. 11, pp. 56–61, 2002. doi : 10.1145/581571.581573.
- [2] Amazon Web Services, “AWS Snowball Documentation,” 2024. Available : <https://aws.amazon.com/snowball/>
- [3] P. Leitner and J. Cito, “Patterns in the Chaos : A Study of Performance Variation and Predictability in Public IaaS Clouds,” in *Proc. ACM Int. Conf. on Cloud Engineering*, 2016, pp. 1–10.
- [4] D. Thain, T. Tannenbaum, and M. Livny, “Distributed Computing in Practice : The Condor Experience,” *Concurrency and Computation : Practice and Experience*, vol. 17, no. 2–4, pp. 323–356, 2005.
- [5] A. S. Tanenbaum and M. Van Steen, *Distributed Systems : Principles and Paradigms*, 2nd ed. Prentice Hall, 2007.
- [6] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.
- [7] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
- [8] B. Burns, J. Beda, and K. Hightower, *Kubernetes : Up and Running*. O’Reilly Media, 2019.
- [9] H. Garcia-Molina, “Elections in a Distributed Computing System,” *IEEE Trans. Computers*, vol. C-31, no. 1, pp. 48–59, 1982.

Annexes